



인하공업전문대학
INHA TECHNICAL COLLEGE

C/C++ 프로그래밍 3주차

인하공업전문대학 컴퓨터정보과
김한결

목차

5.1 제어문

5.2 if 문

5.3 다중 선택문

학습목표

- C 언어에서 쓰이는 제어문의 개념을 이해한다.
- C 언어의 제어문인 if 문과 다중 선택문인 switch case 문에 대해 알아본다.
- 중첩된 조건문을 사용하여 복잡한 논리를 구현하는 방법을 알아본다.

Section 5.1

제어문

1. 제어문

■ 제어문의 개념

- 프로그램에서 명령어의 실행 흐름을 제어하는 문장
 - 특정 상황에서 다른 실행 흐름을 선택
 - 특정 문장 반복 실행
 - 다른 문장으로 이동

1. 제어문

■ 제어문의 개념

■ 조건문

- 주어진 조건의 참, 거짓을 판단하여 결과에 따른 코드 블록을 실행하는 제어문
- 프로그램의 실행 흐름을 분기하여 다양한 상황에 대응하는 로직 구현 가능

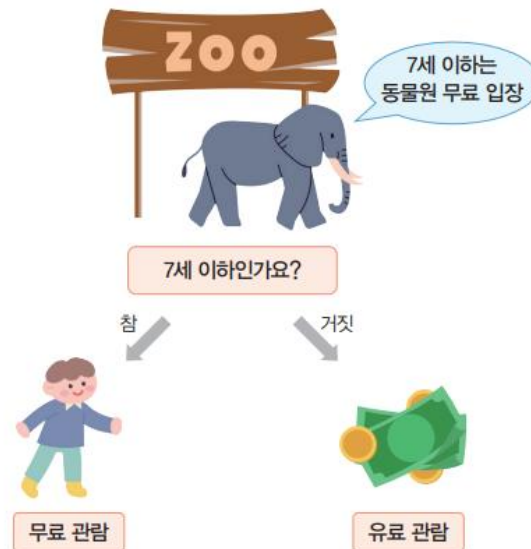


그림 5-1 조건의 참, 거짓에 따른 실행 구조

1. 제어문

■ 제어문의 개념

■ 반복문

- 프로그램에서 특정 코드를 반복하여 실행하는 제어문
- 주어진 조건을 충족하는 동안 코드가 계속 실행되며, 조건을 더 이상 충족하지 않으면 반복을 중단하고 다음 코드 실행

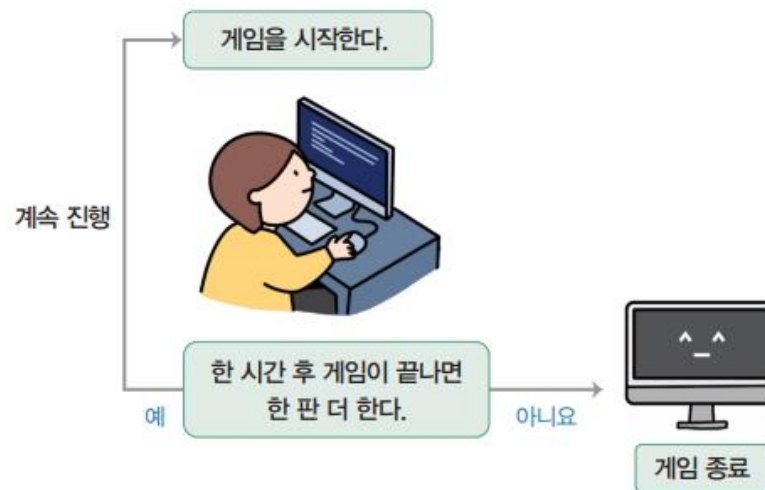


그림 5-2 조건의 판단 결과에 따라 반복을 수행하는 구조

1. 제어문

■ 제어문의 구조

- 순차 구조: 프로그램의 문장이 순서대로 실행되는 구조
- 선택 구조: 프로그램의 흐름을 조건에 따라 다르게 제어하는 구조
- 반복 구조: 특정 조건을 충족할 때까지 반복적으로 실행되는 구조

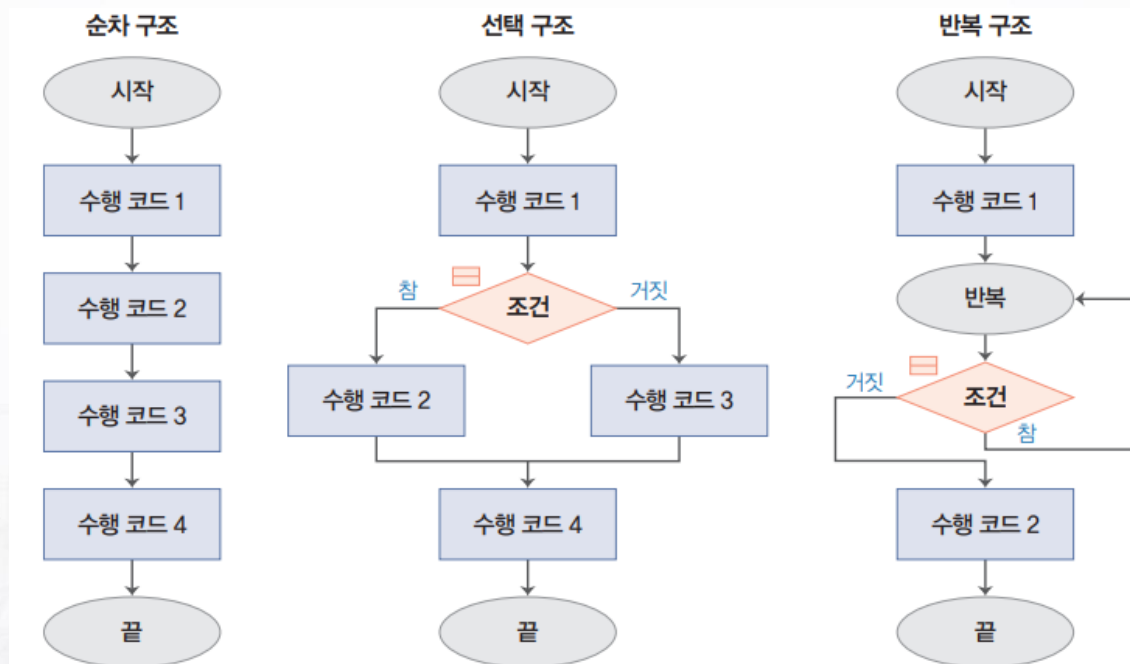


그림 5-3 제어문의 세 가지 구조를 나타낸 순서도

1. 제어문

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① 조건이 참인지 거짓인지 판단하여 명령을 선택적으로 다르게 실행하는 명령문은 ()이다.
- ② 조건이 참인지 거짓인지 판단하여 명령을 반복적으로 실행하는 명령문은 ()이다.

Section 5.2

if 문

2. if 문

■ 단순 if 문

- 조건식의 결과가 참일 때만 특정 코드 블록 실행
- 조건식의 결과가 거짓이면 해당 코드 블록 미실행

| Syntax | 단순 if 문

```
if (조건식)
{
    // 조건식이 참일 때 실행될 코드 블록
}
```

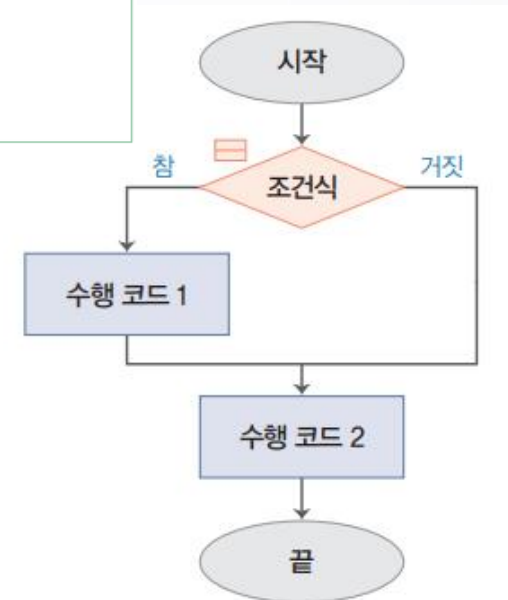


그림 5-4 단순 if 문의 순서도

2. if 문

■ 단순 if 문

■ 단순 if 문의 특징

- 조건식 평가
 - 단순 if 문에서는 조건식을 판단하여 참인지 거짓인지 결정
- 조건이 참일 때만 실행
 - 조건식의 결과가 참이면 if 문에 속한 코드 블록을 실행, 거짓이면 if 문에 속한 코드 블록을 건너뛰고 실행
- 선택적 실행 가능
 - 주어진 조건에 따라 다양한 상황에 대응하는 로직을 구현
- 간결한 구조
 - 단순 if 문은 조건식과 실행할 코드 블록만으로 구성되어 간결하고 명확한 구조

2. if 문

예제 5-1 단순 if 문을 사용하여 변수 증감 여부 판단하기

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int i, a = 10;
07
08     printf("i 값 입력: ");
09     scanf("%d", &i);
10
11     if (i == 3) {
12         a++;
13     }
14
15     printf("판단 이후 a 값은?: %d ", a);
16
17     return 0;
18 }
```

입력된 i 값이 3일 때만 a++가 실행됨

i 값 입력: 3
판단 이후 a 값은? 11

i 값 입력: 2
판단 이후 a 값은? 10

2. if 문

예제 5-2 단순 if 문을 사용하여 정수 판단하기

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int num;
07     printf("정수를 입력하세요: ");
08     scanf("%d", &num);
09     // 10, 13, 16행에서는 if 문에 포함된 코드가 하나이므로 중괄호를 생략함
10     if (num > 0) num이 양수인지 확인
11         printf("%d는 양수입니다.\n", num);
12
13     if (num < 0) num이 음수인지 확인
14         printf("%d는 음수입니다.\n", num);
15
16     if (num == 0) num이 0인지 확인
17         printf("0입니다.\n", num);
18
19     return 0;
20 }
```

정수를 입력하세요: 555
555는 양수입니다.

정수를 입력하세요: 0
0입니다.

정수를 입력하세요: -555
-555는 음수입니다.

2. if 문

■ if~else 문

- 조건식의 결과에 따라 2개의 코드 블록 중 하나를 선택하여 실행

| Syntax | if~else 문

```
if (조건식)
{
    // 조건식이 참일 때 실행될 코드 블록
}
else
{
    // 조건식이 거짓일 때 실행될 코드 블록
}
```

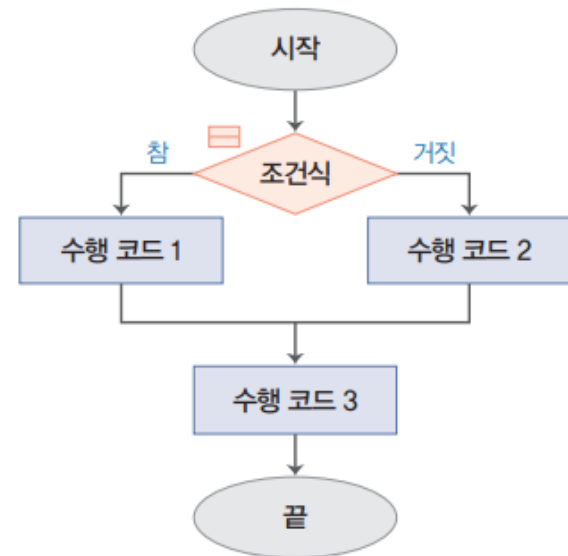


그림 5-5 if~else 문의 순서도

2. if 문

■ if~else 문

▪ if~else 문의 특징

- 조건식에 따른 실행
 - 주어진 조건이 참인지 거짓인지에 따라 다른 코드 블록이 실행
- 대체 실행 경로
 - 조건식이 거짓이면 else 블록에 있는 코드가 실행
- 조건의 다양성
- 코드 블록
 - if와 else 각각의 코드 블록은 여러 문장을 중괄호로 묶어 나타낼 수 있음
- 읽기 쉽고 명확한 코드

2. if 문

예제 5-3 if~else 문을 사용하여 알파벳 출력하기

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int ku;
07     char cd = 'A';
08
09     printf("정수 숫자 입력: ");
10     scanf("%d", &ku);
11
12     if (ku != 100) {
13         cd++;
14         printf("cd: %c", cd);
15     }
16     else {
17         printf("cd: %c", cd);
18     }
19
20     return 0;
21 }
```

입력받은 값이 100이 아닌지 확인

100이 아니면 값을 1 증가시킴

100이면 'A' 출력

정수 숫자 입력: 100
cd: A

정수 숫자 입력: 50
cd: B

2. if 문

예제 5-4 if~else 문을 사용하여 알파벳 문자 변경하기

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03 #include <ctype.h>
04
05 int main()
06 {
07     char ch;
08     printf("알파벳을 입력하세요: ");
09     scanf(" %c", &ch);
10
11     if (ch >= 'a' && ch <= 'z') {
12         ch = (ch - 'a' + 3) % 26 + 'a';
13     }
14     else {
15         ch = (ch - 'A' + 3) % 26 + 'A';
16     }
17
18     printf("변경된 문자: %c\n", ch);
19
20     return 0;
21 }
```

문자 관련 함수를 사용하기 위한 헤더 파일 선언

입력된 문자가 소문자인지 확인

세 칸 뒤로 이동

입력된 문자가 대문자일 때 수행

알파벳을 입력하세요: A
변경된 문자: D

알파벳을 입력하세요: a
변경된 문자: d

2. if 문

■ 다중 if~else 문

- 다양한 조건을 평가하고 여러 코드 블록 중 하나를 선택하여 실행하는 제어문

| Syntax | 다중 if~else 문

```
if (조건식 1)
{
    // 조건식 1이 참일 때 실행될 코드 블록
}
else if (조건식 2)
{
    // 조건식 1이 거짓이고 조건식 2가 참일 때 실행될 코드 블록
}
else
{
    // 조건식 2가 거짓일 때 실행될 코드 블록
}
```

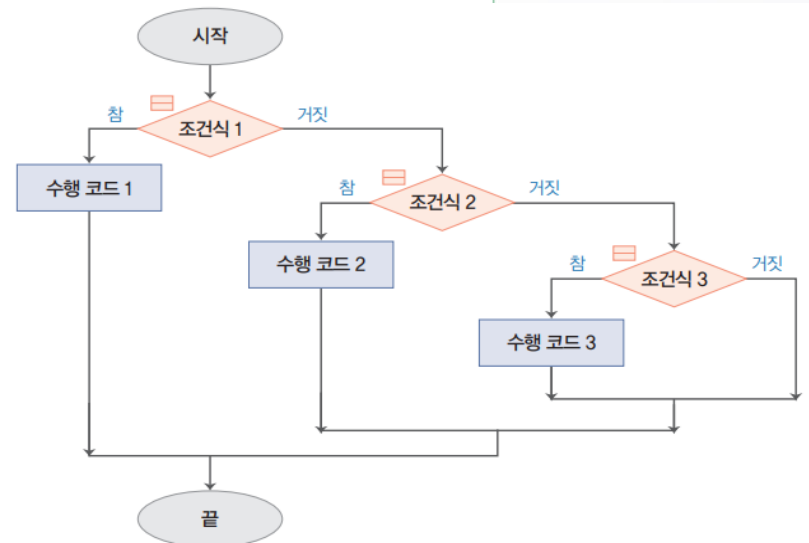


그림 5-6 다중 if~else 문의 순서도

2. if 문

■ 다중 if~else 문

▪ 다중 if~else 문의 특징

- 다중 조건 검사
 - else if 문을 사용하여 여러 조건을 순차적으로 평가할 수 있음
- 선택적 실행
 - else if 문을 사용하여 다양한 조건에 대응하는 다른 코드 블록을 실행
- 순차적 평가
- 간결한 구조

2. if 문

예제 5-5 다중 if~else 문을 사용하여 학점 판별하기

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int outcome;
07     char grades;
08
09     printf("결과 점수를 입력하세요(0-100): ");
10     scanf("%d", &outcome);
11
12     if (outcome >= 90) {
13         grades = 'A';
14     }
15     else if (outcome >= 80) {
16         grades = 'B';
17     }
18     else if (outcome >= 70) {
19         grades = 'C';
20     }
21     else if (outcome >= 60) {
22         grades = 'D';
23     }
24     else {
25         grades = 'F';
26     }
27     printf("학점은: %c \n", grades);
28     return 0;
29 }
```

점수에 따라 학점을 결정하는 다중 if~else 문

결과 점수를 입력하세요(0-100): 95
학점은: A

결과 점수를 입력하세요(0-100): 85
학점은: B

2. if 문

예제 5-6 다중 if~else 문을 사용하여 요금 계산하기

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int age, fee;
07
08     printf("나이를 입력하세요: ");
09     scanf("%d", &age);
10
11     if (age <= 5) {
12         fee = 0;
13     }
14     else if (age <= 12) {
15         fee = 1000;
16     }
17     else if (age <= 18) {
18         fee = 1500;
19     }
20     else if (age <= 60) {
21         fee = 3000;
22     }
23     else {
24         fee = 0;
25     }
26     printf("요금은: %d원\n", fee);
27
28     return 0;
29 }
```

나이에 따라 요금을 결정하는 다중 if~else 문

나이를 입력하세요: 5
요금은: 0원

나이를 입력하세요: 25
요금은: 3000원

2. if 문

■ 중첩 if 문

- if 문 또는 else 문 내부에 다른 if 문 또는 else 문이 포함된 구조
- 단계별로 조건 평가 가능하여 더욱 세밀한 흐름 제어 가능

| Syntax | 중첩 if 문

```
if (조건식 1)
{
    if (조건식 2)
    {
        // 조건식 1과 조건식 2가 모두 참일 때 실행될 코드 블록
    }
    else
    {
        // 조건식 1이 참이고 조건식 2가 거짓일 때 실행될 코드 블록
    }
}
else
{
    if (조건식 3)
    {
        // 조건식 1이 거짓이고 조건식 3이 참일 때 실행될 코드 블록
    }
    else
    {
        // 조건식 1과 조건식 3이 모두 거짓일 때 실행될 코드 블록
    }
}
```

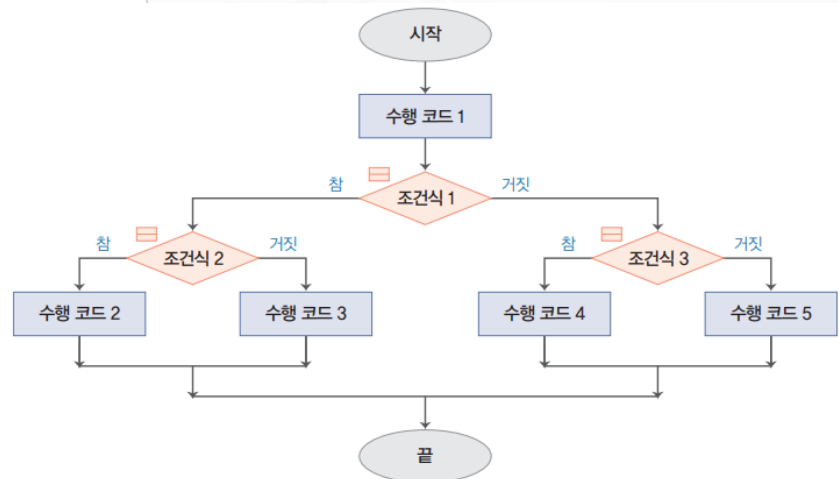


그림 5-7 중첩 if 문의 순서도

2. if 문

■ 중첩 if 문

▪ 중첩 if 문의 특징

- 복잡한 조건 처리
- 순차적 평가
- 구조의 복잡성
- 분기의 다양성
 - 중첩 if 문을 통해 다양한 조건에 따른 실행 경로를 생성

2. if 문

예제 5-7 중첩 if 문을 사용하여 MBTI 판별하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     char first, second; // MBTI의 첫 번째(E/I), 두 번째(N/S) 글자를 저장할 변수
07     printf("MBTI의 첫 번째와 두 번째 글자를 입력하세요 (예: E/I, N/S): ");
08     scanf("%c %c", &first, &second);
09
10     // 첫 번째 글자가 'E'(외향)인지 'I'(내향)인지에 따라 출력
11     if (first == 'E') {
12         printf("당신은 외향적인 성격을 가지고 있습니다.\n");
13         if (second == 'S') {
14             printf("감각형입니다.\n");
15         }
16         else if (second == 'N') {
17             printf("직관형입니다.\n");
18         }
19     }
20     else if (first == 'I') {
21         printf("당신은 내향적인 성격을 가지고 있습니다.\n");
22         if (second == 'S') {
23             printf("감각형입니다.\n");
24         }
25         else if (second == 'N') {
26             printf("직관형입니다.\n");
27         }
28     }
```

첫 번째, 두 번째 글자 판별

첫 번째 글자가 E이면 '당신은 외향적인 성격을 가지고 있습니다.'를 출력

두 번째 글자가 S이면 '감각형입니다.'를 출력하고, 그렇지 않으면 '직관형입니다.'를 출력

첫 번째 글자가 I이면 '당신은 내향적인 성격을 가지고 있습니다.'를 출력

MBTI의 첫 번째와 두 번째 글자를 입력하세요 (예: E/I, N/S): I N
당신은 내향적인 성격을 가지고 있습니다.
직관형입니다.
MBTI의 세 번째와 네 번째 글자를 입력하세요 (예: F/T, P/J): F J
감정적이고 사람 중심적으로 판단하는 감정형입니다.
계획형입니다.

```
29
30     char third, fourth; // MBTI의 세 번째(T/F), 네 번째(J/P) 글자를 저장할 변수
31
32     printf("MBTI의 세 번째와 네 번째 글자를 입력하세요 (예: F/T, P/J): ");
33     scanf("%c %c", &third, &fourth);
34
35     // 세 번째 글자가 'T'(사고형)인지 'F'(감정형)인지에 따라 출력
36     if (third == 'T') {
37         printf("논리적이고 객관적으로 판단하는 사고형입니다.\n");
38         if (fourth == 'J') {
39             printf("계획형입니다.\n");
40         }
41         else if (fourth == 'P') {
42             printf("즉흥적입니다.\n");
43         }
44     }
45     else if (third == 'F') {
46         printf("감정적이고 사람 중심적으로 판단하는 감정형입니다.\n");
47         if (fourth == 'J') {
48             printf("계획형입니다.\n");
49         }
50         else if (fourth == 'P') {
51             printf("즉흥적입니다.\n");
52         }
53     }
54     return 0; // 프로그램 종료
55 }
```

세 번째, 네 번째 글자 판별

세 번째 글자가 T이면 '논리적이고 객관적으로 판단하는 사고형입니다.'를 출력

네 번째 글자가 J이면 '계획형입니다.'를 출력하고, 그렇지 않으면 '즉흥적입니다.'를 출력

세 번째 글자가 F이면 '감정적이고 사람 중심적으로 판단하는 감정형입니다.'를 출력

2. if 문

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

if~else 문은 조건식의 결과가 참이면 if 문에 속한 코드 블록이 실행되고, 거짓이면 () 문에 속한 코드 블록이 실행된다.

2 중첩 if 문의 특징에 대한 설명으로 옳지 않은 것을 고르시오.

- ① 중첩 if 문을 사용하면 여러 조건을 복합적으로 검사하고 조건에 따른 다양한 작업을 수행할 수 있다.
- ② 중첩 if 문의 조건은 순차적으로 평가되며, 상위 if 문의 조건이 거짓이면 내부에 포함된 if 문이 실행될 수 있다.
- ③ 중첩 if 문을 과도하게 사용하면 코드의 가독성이 떨어질 수 있어 가능한 한 깊이를 최소화하는 것이 좋다.
- ④ 중첩 if 문을 사용하면 다양한 조건에 따른 실행 경로를 생성할 수 있다.

2. if 문

LAB 윤년 계산하기

■ 실행 결과

연도를 입력하세요 : 2024
윤년입니다.

연도를 입력하세요 : 2023
평년입니다.



2. if 문

LAB 윤년 계산하기

■ 조건

- 연도를 입력받는다.
- 윤년인지 판별하는 if 문을 작성한다. AND 연산자(&&)보다 OR 연산자(||)의 우선순위가 높으므로 괄호를 사용하여 조건식을 작성한다.
- if~else 문으로 작성한다.

■ 힌트

윤년을 계산하는 규칙은 다음과 같다.

- 연도가 4로 나누어떨어지는 해는 윤년이다.
- 연도가 4로 나누어떨어지면서 100으로도 나누어떨어지는 해는 윤년이 아니다.
- 연도가 400으로 나누어떨어지는 해는 윤년이다.

2. if 문

LAB 윤년 계산하기

문제 해결

LAB_5-1.c

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main(void)
05 {
06     int year; // 입력받을 연도를 저장할 변수 선언
07
08     printf("연도를 입력하세요 : ");
09     scanf("%d", &year);
10
11     if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
12         printf("윤년입니다\n"); // 윤년을 판별하는 조건문
13     }
14     else {
15         printf("평년입니다\n"); // 위의 조건이 모두 거짓일 때 실행
16     }
17
18     return 0;
19 }
```

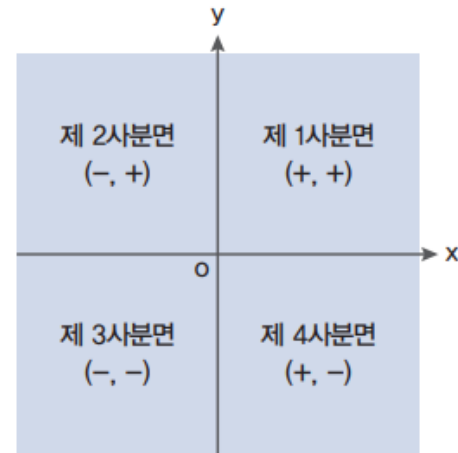
2. if 문

LAB 좌표 위치 확인하기

■ 실행 결과

```
x 좌표를 입력하세요: 5
y 좌표를 입력하세요: -2
(5, -2) 좌표는 제 4사분면에 있습니다.
```

```
x 좌표를 입력하세요: -2
y 좌표를 입력하세요: 4
(-2, 4) 좌표는 제 2사분면에 있습니다.
```



■ 조건

- x 좌표와 y 좌표를 입력받는다.
- 좌표를 판별하는 if 문을 작성할 때 AND 연산자를 사용하여 조건식을 구성한다.

2. if 문

LAB 좌표 위치 확인하기

문제 해결

LAB_5-2.c

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int x, y;
07
08     printf("x 좌표를 입력하세요: ");
09     scanf("%d", &x);
10     printf("y 좌표를 입력하세요: ");
11     scanf("%d", &y);
12
13     if (x > 0 && y > 0) {
14         printf("(%d, %d) 좌표는 제 1사분면에 있습니다.\n", x, y);
15     }
16     else if (x < 0 && y > 0) {
17         printf("(%d, %d) 좌표는 제 2사분면에 있습니다.\n", x, y);
```

x, y 좌표 입력받기

x, y 좌표가 모두 양수일 때

x 좌표는 음수, y 좌표는 양수일 때

2. if 문

LAB 좌표 위치 확인하기

```
18     }
19     else if (x < 0 && y < 0) {
20         printf("(%d, %d) 좌표는 제 3사분면에 있습니다.\n", x, y);
21     }
22     else if (x > 0 && y < 0) {
23         printf("(%d, %d) 좌표는 제 4사분면에 있습니다.\n", x, y);
24     }
25     else if (x == 0 && y == 0) {
26         printf("(%d, %d) 좌표는 원점입니다.\n", x, y);
27     }
28     else if (x == 0) {
29         printf("(%d, %d) 좌표는 y 축 위에 있습니다.\n", x, y);
30     }
31     else {
32         printf("(%d, %d) 좌표는 x 축 위에 있습니다.\n", x, y);
33     }
34
35     return 0;
36 }
```

x, y 좌표가 모두 음수일 때

x 좌표는 양수, y 좌표는 음수일 때

x, y 좌표가 모두 0일 때

x 좌표가 0일 때

Section 5.3

다중 선택문

3. 다중 선택문

■ switch case 문

- 하나의 변수나 표현식의 값에 따라 여러 가지 선택 사항 중 하나를 선택하여 실행할 때 사용
- 값이나 상태에 따라 각기 다른 코드 블록을 실행해야 할 때 유용

| Syntax | switch case 문

```
switch (표현식)
{
    case 1:
        // case 1에 해당할 때 실행될 코드
        break;
    case 2:
        // case 2에 해당할 때 실행될 코드
        break;
    ...
    default:
        // 어떤 case에도 해당하지 않을 때 실행될 코드
}
```

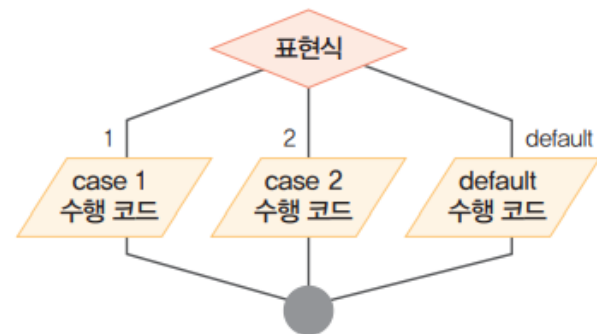


그림 5-8 switch case 문의 순서도

3. 다중 선택문

■ switch case 문

▪ switch case 문의 특징

- 값에 따른 분기
 - 표현식의 결과값과 일치하는 case 블록을 실행
- 정수 및 열거형 값만 가능
- case 값의 유일성
 - 중복된 case 값이 있으면 컴파일 오류가 발생
- break 문의 중요성
 - case 블록 끝에 break 문을 사용하여 해당 case 블록의 코드 실행을 중지하고 switch 문을 종료
- default 절
 - 일반적으로 default 절에는 break 문을 작성하지 않음

3. 다중 선택문

예제 5-8 switch 문을 사용하여 음료 자판기 상태 확인하기

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int currentState;
07
08     printf("\n현재 상태 (0: 동전 투입, 1: 음료 선택, 2: 음료 제공, 3: 대기로 돌아가기) : ");
09     scanf("%d", &currentState);
10
11     switch (currentState){
12     case 0: case 값에 따라 출력할 메시지 지정
13         printf("대기 중\n");
14         printf("0. 동전 투입\n");
15         break; 각 case 코드의 실행을 마무리하고 switch 블록을 빠져나옴
16     case 1:
17         printf("동전 투입 상태\n");
18         printf("1. 음료 선택\n");
19         break;
20     case 2:
21         printf("음료 선택 중\n");
22         printf("2. 음료 제공\n");
23         break;
24     case 3:
25         printf("음료 제공 중\n");
26         printf("3. 대기 상태로 돌아가기\n");
27         break;
28     default:
29         printf("알 수 없는 상태\n");
30         return 1; 비정상적 프로그램 종료
31     }
32     return 0;
33 }
```

```
현재 상태 (0: 동전 투입, 1: 음료 선택, 2: 음료 제공, 3: 대기로 돌아가기) : 0
대기 중
0. 동전 투입
```

3. 다중 선택문

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① () 문은 case 블록 끝에 작성하며, 해당 case 블록의 코드 실행을 중지하고 switch 문을 종료하는 기능을 한다. default 절에는 () 문을 작성하지 않는다.
- ② switch 문의 조건식과 case 값으로는 정수와 문자만 가능하고 ()를 사용하면 오류가 발생한다.

3. 다중 선택문

LAB 로봇 시뮬레이션 프로그램 만들기

▪ 실행 결과

로봇 시작 상태입니다.

(1) 이동 상태로 변경

상태 입력>> 1

입력하신 번호: 1

로봇 이동 중입니다.

(2) 작업 상태로 변경

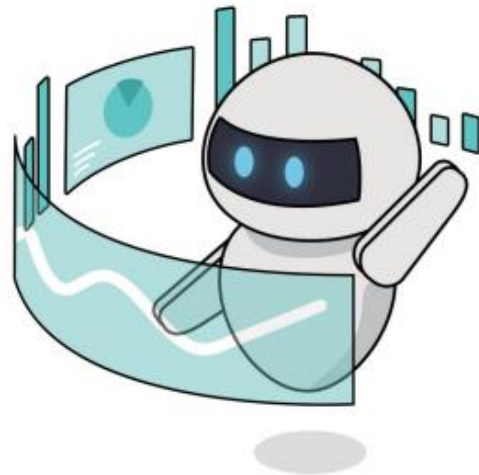
로봇 시작 상태입니다.

(1) 이동 상태로 변경

상태 입력>> 5

입력하신 번호: 5

잘못된 입력입니다. 다시 입력하세요.



3. 다중 선택문

LAB 로봇 시뮬레이션 프로그램 만들기

■ 조건

- 로봇은 시작 상태에서 출발한다.
- 로봇을 이동 상태로 전환할 수 있다. 이동 상태에서는 작업 상태로 전환할 수 있고, 작업 상태에서는 종료 상태로 전환 할 수 있다.
- 사용자는 각 상태에서 특정한 입력을 통해 로봇을 다음 상태로 전환한다. 즉 각 상태에서 전환이 가능한 다음 상태의 번호를 입력한다.

■ 힌트

- 각 상태에서 잘못된 번호가 입력되었을 때의 처리 방법을 고려한다.

3. 다중 선택문

LAB 로봇 시뮬레이션 프로그램 만들기

문제 해결

LAB_5-3.c

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 #define START 0
05 #define MOVE 1
06 #define WORK 2
07 #define STOP 3
```

```
08
09 int main() {
10     int currentState = START; // 로봇의 초기 상태를 START로 설정
11     int input; // 사용자의 입력을 받을 변수
12
13     while (1) {
14         switch (currentState) { // 현재 상태에 따라 다른 동작을 수행
15             case START:
16                 printf("로봇 시작 상태입니다.\n");
17                 printf("(1) 이동 상태로 변경\n");
18                 printf("상태 입력>> ");
19                 scanf("%d", &input); // 사용자의 입력으로 로봇의 상태 전환
20                 printf("입력하신 번호: %d\n", input); // 입력된 번호 출력
21                 if (input == 1) {
22                     currentState = MOVE; // START 상태에서 1을 입력받으면 MOVE로 전환
23                 }
24             else {
25                 printf("잘못된 입력입니다. 다시 입력하세요.\n");
26             }
27             break;

```

3. 다중 선택문

LAB 로봇 시뮬레이션 프로그램 만들기

```
28 case MOVE:
29     printf("로봇 이동 중입니다.\n");
30     printf("(2) 작업 상태로 변경\n");
31     printf("상태 입력>> ");
32     scanf("%d", &input);
33     printf("입력하신 번호: %d\n", input);    // 입력된 번호 출력
34     if (input == 2) {
35         currentState = WORK;
36     }
37     else {
38         printf("잘못된 입력입니다. 다시 입력하세요.\n");
39     }
40     break;
```

MOVE 상태에서 2를 입력받으면 WORK로 전환

```
41 case WORK:
42     printf("로봇 작업 중입니다.\n");
43     printf("(3) 종료 상태로 변경");
44     printf("상태 입력>> ");
45     scanf("%d", &input);
46     printf("입력하신 번호: %d\n", input);    // 입력된 번호 출력
47     if (input == 3) {
48         currentState = STOP;
49     }
50     else {
51         printf("잘못된 입력입니다. 다시 입력하세요.\n");
52     }
53     break;
54 case STOP:
55     printf("로봇이 종료되었습니다.\n");
56     return 0;
57 default:
58     printf("알 수 없는 상태입니다.\n");
59     return 1;
60 }
61 }
62 return 0;
63 }
```

WORK 상태에서 3을 입력받으면 STOP으로 전환

STOP 상태가 되면 프로그램 종료

비정상적 프로그램 종료

목차

6.1 반복문의 개요

6.2 while 문

6.3 do~while 문

6.4 for 문

6.5 분기문

학습목표

- C 언어에서 쓰이는 반복문의 개념과 필요성을 이해한다.
- C 언어의 반복문인 while 문, do~while 문, for 문의 구조와 동작 방식을 알아본다.
- C 언어의 분기문인 break 문, continue 문에 대해 알아본다.

Section 6.1

반복문의 개요

1. 반복문의 개요

■ 반복문의 개념

- 특정 작업을 여러 번 반복적으로 실행할 때 사용하는 명령문
- 데이터 목록을 처리하거나, 같은 작업을 정해진 횟수만큼 반복하거나, 특정 조건을 충족할 때까지 계속해서 작업을 수행해야 할 때 유용
- 같은 코드를 여러 번 쓰지 않고도 동일한 작업 반복 수행 가능

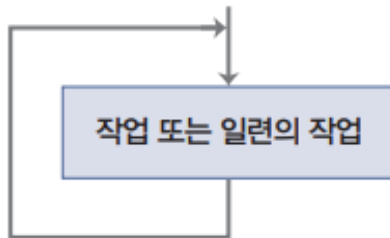


그림 6-1 반복의 개념



그림 6-2 쿠키 조리법에 비유되는 반복문

1. 반복문의 개요

■ 반복문의 특징

- C 언어에서 제공하는 반복문은 while 문, do~while 문, for 문
- 특정 상황과 요구 사항에 따라 적절한 것을 사용
- 반복문을 올바르게 사용하면 코드의 효율성과 가독성이 크게 향상
- 잘못 사용하면 프로그램의 성능이 떨어지고 무한 반복과 같은 문제를 초래

1. 반복문의 개요

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

C 언어에서 제공하는 반복문은 (), (), ()이다.

Section 6.2

while 문

2. while 문

■ while 문의 구조

- 주어진 조건이 참인 동안 특정 코드 블록을 반복적으로 실행하는 제어 명령문
- 반복문을 시작하기 전에 조건을 검사하는 수행 전 검사 루프 구조로 초깃값 설정
- 조건식을 선언하여 반복 여부 결정
- 코드 블록 내에 증감식을 작성하여 반복 횟수를 제어

| Syntax | while 문

```
while (조건식)
{
    // 조건식이 참일 때 실행될 코드
}
```

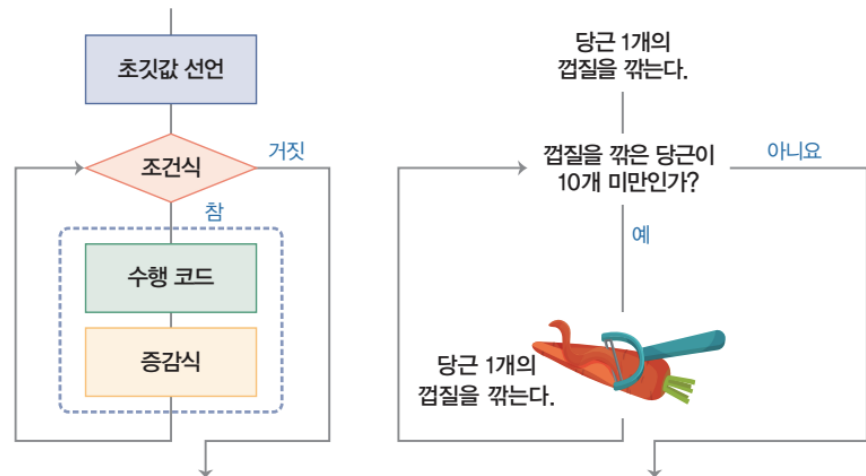


그림 6-3 while 문의 실행 흐름

2. while 문

■ while 문의 동작 원리

- **조건식:** 반복문 실행 여부를 결정하는 조건식 평가
- **코드 실행:** 조건식의 결과가 참이면 중괄호 안의 수행 코드 실행
- **조건식 평가 반복:** 코드 실행 후 조건식으로 되돌아가 다시 평가하고, 참이면 수행 코드 다시 실행
- **프로그램 종료:** 위 단계를 반복하다 조건식의 결과가 거짓이면 반복문 해제

```
int count = 0;           // 초기값
while (count < 5) {      // 반복 조건문
    printf("count: %d\n", count);
    count++;             // 증감식
}
```

2. while 문

예제 6-1 while 문을 사용하여 숫자 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int n, lineCnt;
07
08     printf("1~100 사이 숫자 입력: ");
09     scanf("%d", &n);
10
11     if (n > 100) n = 100;
12
13     lineCnt = 0;
14     while (n > 0) {
15         if (lineCnt < 10) {
16             lineCnt++;
17         }
18         else {
19             printf("\n");
20             lineCnt = 1;
21         }
22         printf("%3d ", n--);
23     }
24 }
```

100을 초과하는 숫자가 입력되면 n을 100으로 조정

n이 0보다 작으면 다음 작업을 반복

lineCnt가 10보다 작으면 lineCnt를 1 증가시킴

lineCnt가 10이면 줄 바꿈(\n) 후 lineCnt를 1로 초기화

n의 값을 세 자리 공간에 오른쪽 정렬로 출력하고 n의 값을 1 감소시킴

1~100 사이 숫자 입력: 50

50	49	48	47	46	45	44	43	42	41
40	39	38	37	36	35	34	33	32	31
30	29	28	27	26	25	24	23	22	21
20	19	18	17	16	15	14	13	12	11
10	9	8	7	6	5	4	3	2	1

2. while 문

예제 6-2 while 문을 사용하여 정수의 각 자릿수 개수와 합 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int number, count = 0, sum = 0;
07
08     printf("\n정수 입력: ");
09     scanf("%d", &number);
10     printf("당신이 입력한 수: %d\n\n", number);
11
12     while (number != 0) {
13         count++;
14         sum += number % 10;
15         number /= 10;
16     }
17     printf("정수의 각 자릿수 개수: %d\n", count);
18     printf("입력한 숫자의 합계: %d\n", sum);
19
20     return 0;
21 }
```

number가 0이 아닌 동안 다음 작업을 반복

```
정수 입력: 12345
당신이 입력한 수: 12345

정수의 각 자릿수 개수: 5
입력한 숫자의 합계: 15
```

2. while 문

예제 6-3 while 문을 사용하여 여러 번 입력된 정수의 합 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int n = 0, sum = 0;
07
08     printf("숫자 입력(종료:ctrl+z) : \n");
09
10     while (scanf("%d", &n) != EOF) {
11         sum = sum + n;
12     }
13     printf("\n 합계 : %d \n", sum);
14
15     return 0;
16 }
```

종료 방법인 **Ctrl + Z** 안내

scanf() 함수의 반환값이 EOF가 아닌 동안
다음 작업을 반복

숫자 입력(종료:ctrl+z) :

1
2
3
4
5

^Z
^Z
^Z

Ctrl + Z + Enter

합계 : 15

2. while 문

확인문제

1 while 문의 동작 원리에 따라 다음 설명을 순서대로 나열하시오.

- ㉠ 반복 수행을 하다 조건식의 결과가 거짓이면 반복문을 빠져나온다.
- ㉡ 수행 코드를 실행한 후 조건식으로 되돌아가 다시 평가하고, 결과가 참이면 수행 코드를 다시 실행한다.
- ㉢ 조건식의 결과가 참이면 중괄호 안의 수행 코드를 실행한다.
- ㉣ 조건식을 평가한다.

2 윈도우 환경에서 '파일의 끝'을 의미하는 EOF의 키 조합은 무엇인가?

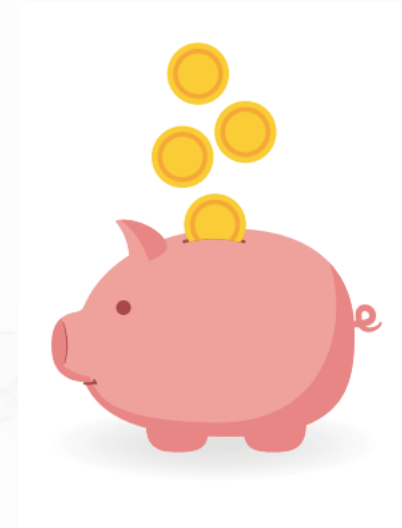
- ① **Ctrl** + **C**
- ② **Ctrl** + **V**
- ③ **Ctrl** + **Z**
- ④ **Alt** + **T**

2. while 문

LAB 저축 계획 세우기

■ 실행 결과

```
월급을 입력하세요(만 원 단위로 입력하세요): 300  
월간 지출을 입력하세요(만 원 단위로 입력하세요): 100  
목표 금액을 입력하세요(만 원 단위로 입력하세요): 1000  
  
월급 300만 원  
월간 지출 100만 원  
목표 금액 1000만 원  
  
# 목표 금액을 달성하는 데 5개월이 걸립니다.
```



■ 힌트

- 현재까지 저축한 금액이 목표 금액보다 적으면 루프를 계속 반복한다.
while (현재까지 저축한 금액 < 목표 금액)

2. while 문

LAB 저축 계획 세우기

문제 해결

LAB_6-1.c

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int salary, monthlyExpense, targetAmount; // 월급, 월간 지출, 목표 금액
07     int totalSavings = 0;                      // 저축액=월급-월간 지출
08     int months = 0;                            // 개월 수
09
10     printf("월급을 입력하세요(만 원 단위로 입력하세요): ");
11     scanf("%d", &salary);
12
13     printf("월간 지출을 입력하세요(만 원 단위로 입력하세요): ");
14     scanf("%d", &monthlyExpense);
15
16     printf("목표 금액을 입력하세요(만 원 단위로 입력하세요): ");
17     scanf("%d", &targetAmount);
18
19     while (totalSavings < targetAmount) {
20         totalSavings += (salary - monthlyExpense);
21         months++;
22     }
23
24     printf("\n월급 %d만 원\n", salary);
25     printf("월간 지출 %d만 원\n", monthlyExpense);
26     printf("목표 금액 %d만 원\n", targetAmount);
27     printf("# 목표 금액을 달성하는 데 %d개월이 걸립니다.\n", months);
28
29     return 0;
30 }
```

저축 금액이 목표 금액보다 적은 동안 반복

한 달이 지났으므로 개월 수를 증가시킴

Section 6.3

do~while 문

3. do~while 문

■ do~while 문의 구조

- 주어진 조건이 참인 동안 블록 내부의 코드를 반복적으로 실행하는 제어 명령문
- while 문과 달리 do~while 문에 진입하면 무조건 블록 내부의 수행 코드를 실행

| Syntax | do~while 문

```
do
{
    // 반복적으로 실행될 코드
} while (조건식);
```

세미콜론 작성

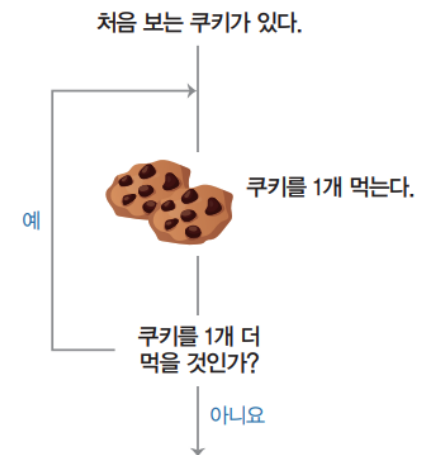
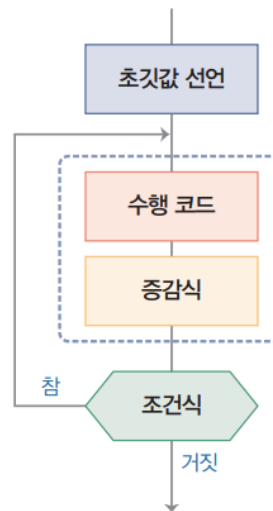


그림 6-4 do~while 문의 실행 흐름

3. do~while 문

■ do~while 문의 동작 원리

- **코드 실행:** do 키워드 다음의 수행 코드를 먼저 실행
- **조건식:** 수행 코드와 증감식 실행 후 while 문의 조건식 평가
- **조건식의 결과에 따른 코드 실행:**
 - 참: 수행 코드를 다시 실행한 후 조건식 재평가
 - 거짓: do~while 문을 종료하고 중괄호 밖의 명령문 실행

```
int count = 0;           // 초기값
do
{
    printf("This will run at least once.\n");
    count++;             // 증감식
} while (count < 5);     // 반복 조건문
```

3. do~while 문

예제 6-4 do~while 문을 사용하여 숫자 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int n, lineCnt;
07
08     printf("1~100 사이 숫자 입력: ");
09     scanf("%d", &n);
10
11     if (n > 100) n = 100;
12
13     lineCnt = 0;
14     do {
15         if (lineCnt < 10) {
16             lineCnt++;
17         }
18         else {
19             printf("\n");
20             lineCnt = 1;
21         }
22         printf("%3d ", n--);
23     } while (n > 0);
24 }
```

n이 100보다 크면 n의 값을 100으로 변경

lineCnt가 10보다 작으면 lineCnt를 1 증가시킴

lineCnt가 10이면 줄 바꿈(\n) 후 lineCnt를 1로 초기화

1~100 사이 숫자 입력: 50

50	49	48	47	46	45	44	43	42	41
40	39	38	37	36	35	34	33	32	31
30	29	28	27	26	25	24	23	22	21
20	19	18	17	16	15	14	13	12	11
10	9	8	7	6	5	4	3	2	1

3. do~while 문

예제 6-5 do~while 문을 사용하여 성적 등급 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int score;
07     char choice;
08     // 사용자의 선택을 저장하는 문자 변수로, 반복 여부를 결정할 때 사용
09     do {
10         printf("성적을 입력하세요 (0~100): ");
11         scanf("%d", &score);
12         // 성적 등급을 판별하는 조건식
13         if (score >= 90) {
14             printf("등급: A\n");
15         }
16         else if (score >= 80) {
17             printf("등급: B\n");
18         }
19         else if (score >= 70) {
20             printf("등급: C\n");
21         }
22         else if (score >= 60) {
23             printf("등급: D\n");
24         }
25         else {
26             printf("등급: F\n");
27         }
28
29         printf("다른 성적을 입력하시겠습니까? (Y/N): ");
30         scanf("%c", &choice);
31         // scanf()에서 [Enter]를 입력받았을 때 남은 버퍼를 비우기 위한 공백
32     } while (choice == 'Y' || choice == 'y');
33     // 사용자가 Y 또는 y를 입력하면 do~while 문을 반복
34
35     return 0;
36 }
```

성적을 입력하세요 (0~100): 85
등급: B
다른 성적을 입력하시겠습니까? (Y/N): Y
성적을 입력하세요 (0~100): 90
등급: A
다른 성적을 입력하시겠습니까? (Y/N): N

3. do~while 문

확인문제

1 다음 코드에서 오류가 발생하는 부분을 찾아 바르게 수정하고 그 이유를 설명하시오.

```
int count = 0;
do {
    printf("This will run at least once.\n");
    count++;
} while (count < 10)
```

Section 6.4

for 문

4. for 문

■ for 문의 구조

- 주어진 조건이 참인 동안 특정 코드 블록을 반복적으로 실행하는 반복 제어문
- 반복을 위한 초기값, 조건식 평가, 반복을 제어하는 증감식 연산이 한 줄에 모두 포함된 간결한 문법

| Syntax | for 문

```
for (초기화; 조건식; 증감식)
{
    // 반복적으로 실행될 코드
}
```



그림 6-5 for 문의 동작 흐름

4. for 문

■ for 문의 동작 원리

- 초기화: 반복 횟수를 저장할 변수를 선언하거나 초기화하는 단계
- 조건식: 결과가 참이면 중괄호 안의 수행 코드 블록 실행, 거짓이면 반복 종료
- 증감식: 반복문의 변수값을 증가시키거나 감소시킬 때 사용

```
int i = 0;
for (i = 0; i < 10; i++) {
    printf("%d\n", i);
}
```

```
int i = 0;
for (i = 0; i < 10; i++) {
    printf("%d\n", i);
}
```

그림 6-6 for 문의 실행 순서

- ① i를 0으로 초기화
- ② 조건식(i < 10) 평가
 - 조건식의 결과가 참이면 ③을 실행
 - 조건식의 결과가 거짓이면 블록을 탈출
- ③을 실행하고 ④ i++를 실행
- ②부터 다시 반복 수행

4. for 문

예제 6-6 for 문을 사용하여 반복 횟수 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int i, limit;
07
08     printf("반복 횟수를 입력: ");
09     scanf("%d", &limit);
10
11     for (i = 1; i <= limit; i++){
12         printf(" %d \n", i);
13     }
14
15     return 0;
16 }
```

1부터 limit의 값까지 1씩 증가시키며 반복

반복문 내에서 현재 i의 값을 출력

반복 횟수를 입력: 3

1
2
3

4. for 문

예제 6-7 for 문을 사용하여 정수의 제곱 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int n;
07
08     printf("정수를 입력: ");
09     scanf("%d", &n);
10
11     for (int i = 1; i <= n; i++) {
12         printf("%d의 제곱: %d\n", i, i * i);
13     }
14
15     return 0;
16 }
```

현재 i의 값과 그 제곱을 출력

```
정수를 입력: 5
1의 제곱: 1
2의 제곱: 4
3의 제곱: 9
4의 제곱: 16
5의 제곱: 25
```

4. for 문

예제 6-8 for 문을 사용하여 A부터 Z까지 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     char ch;
07
08     for (ch = 'A'; ch <= 'Z'; ch++) {
09         printf("%c ", ch);
10     }
11
12     return 0;
13 }
```

반복문에서 사용될 대문자 카운터 변수로 작용하는 문자형 변수

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

4. for 문

■ 다중 for 문의 구조

- for 문 안에 또 다른 for 문이 포함된 구조
- 외부 반복문이 내부 반복문보다 먼저 시작
 - 외부 반복문의 각 반복마다 내부 반복문이 전체적으로 실행
 - 내부 반복문이 종료 되면 외부 반복문의 증감식이 실행되고 다시 내부 반복문이 시작
- 외부 반복문의 조건식이 거짓이 될 때까지 반복

| Syntax | 다중 for 문

```
for (초기화; 조건식; 증감식)
{
    // 외부 반복문에서 실행될 코드
    for (초기화 2; 조건식 2; 증감식 2)
    {
        // 내부 반복문에서 실행될 코드
    }
}
```

4. for 문

■ 다중 for 문의 동작 원리

- 중첩 반복문을 사용하여 # 기호로 이루어진 특정 패턴을 출력하는 예제

개수를 증가시켜 출력하기

ex_multiple.for.c

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int i, j;
06     for (i = 0; i <= 5; i++){
07         for (j = 0; j <= i; j++){
08             printf("#");
09         }
10         printf("\n");
11     }
12     return 0;
13 }
```

외부 반복문의 제어 변수 i, 내부 반복문의 제어 변수 j 선언

출력될 패턴의 행수를 결정하기 위한 외부 반복문

각 행마다 출력될 문자의 수를 결정하기 위한 내부 반복문

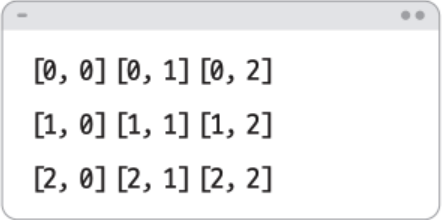
각 행의 문자 출력이 끝나면 줄 바꿈

```
#
##
###
####
#####
#####
```

4. for 문

예제 6-9 다중 for 문을 사용하여 2차원 배열의 요소 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int i, j;
07     for (i = 0; i < 3; i++) {           // 외부 반복문
08         for (j = 0; j < 3; j++) {       // 내부 반복문
09             printf("[%d, %d]", i, j);   // 행렬의 인덱스 출력
10         }
11         printf("\n"); // 각 행이 끝날 때마다 줄 바꿈
12     }
13     return 0;
14 }
```



```
[0, 0] [0, 1] [0, 2]
[1, 0] [1, 1] [1, 2]
[2, 0] [2, 1] [2, 2]
```

4. for 문

예제 6-10 다중 for 문을 사용하여 구구단 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int i, j;
07
08     for (i = 2; i <= 9; i++) {
09         for (j = 1; j <= 9; j++) {
10             printf("%d * %d = %2d \n", i, j, i * j);
11         }
12     }
13     return 0;
14 }
```

2부터 9까지 1씩 증가시키며 단을 반복하는 외부 반복문

1부터 9까지 각 단에서의 곱셈 연산을 반복하는 내부 반복문

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
...
9 * 7 = 63
9 * 8 = 72
9 * 9 = 81
```

4. for 문

예제 6-11 다중 for 문을 사용하여 구구단을 가로로 출력하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int i, j;
07
08     for (i = 1; i <= 9; i++) {
09         for (j = 2; j <= 9; j++) {
10             printf("%d * %d = %2d ", j, i, i * j);
11         }
12         printf("\n");
13     }
14     return 0;
15 }
```

2부터 9까지 1씩 증가시키며 단을 반복하는 외부 반복문

1부터 9까지 각 단에서의 곱셈 연산을 반복하는 내부 반복문

```
2*1= 2 3*1= 3 4*1= 4 5*1= 5 6*1= 6 7*1= 7 8*1= 8 9*1= 9
2*2= 4 3*2= 6 4*2= 8 5*2=10 6*2=12 7*2=14 8*2=16 9*2=18
...
2*8=16 3*8=24 4*8=32 5*8=40 6*8=48 7*8=56 8*8=64 9*8=72
2*9=18 3*9=27 4*9=36 5*9=45 6*9=54 7*9=63 8*9=72 9*9=81
```

4. for 문

확인문제

- 1 for 문을 사용하여 알파벳을 역순으로 출력하는 코드를 작성하시오.
- 2 for 문을 사용하여 알파벳을 역순으로 출력하는 코드를 작성하시오. 단, 아스키코드를 이용하시오.
- 3 다중 for 문을 사용하여 다음과 같은 결과가 출력되는 코드를 작성하시오.

```
#####  
#####  
#####  
#####  
#####  
#####  
#####
```

4. for 문

LAB 계좌 이자 계산하기

■ 실행 결과

```
초기 예금액을 입력하세요: 3000000
연 이자율을 입력하세요 (퍼센트): 3.0
저축 기간을 입력하세요 (년): 5
1년: 예금액= 3090000.00, 이자= 90000.00
2년: 예금액= 3182700.00, 이자= 92700.00
3년: 예금액= 3278181.00, 이자= 95481.00
4년: 예금액= 3376526.43, 이자= 98345.43
5년: 예금액= 3477822.22, 이자= 101295.79
```



4. for 문

LAB 계좌 이자 계산하기

■ 조건

- 초기 예금액, 연 이자율(퍼센트), 저축 기간(년)을 입력받는다.
- 퍼센트인 연 이자율을 소수로 변환하여 계산에 사용한다.
- 초기 예금액에 연 이자율을 곱하여 각 연도의 이자를 계산한다.
이자 = (현재 연도의 예금액 * 연 이자율 / 100)
- 계산된 이자와 갱신된 예금액을 연도별로 소수점 아래 둘째 자리까지 출력한다.

■ 힌트

- 현재 연도가 저축 기간보다 작은 동안 연도를 1씩 증가시키면서 반복한다.
for (연도 = 1; 연도 <= 저축기간; 연도++)

4. for 문

LAB 계좌 이자 계산하기

문제 해결

LAB_6-2.c

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     double initialDeposit;           // 초기 예금액
07     double annualInterestRate, interest; // 연 이자율
08     int year, numYears;               // 저축 기간(년)
09
10     printf("초기 예금액을 입력하세요: ");
11     scanf("%lf", &initialDeposit);
12
13     printf("연 이자율을 입력하세요 (퍼센트): ");
14     scanf("%lf", &annualInterestRate);
15
16     printf("저축 기간을 입력하세요 (년): ");
17     scanf("%d", &numYears);
18
19     // 저축 기간에 따른 예금액과 이자를 계산하고 출력하는 for 문
20     for (year = 1; year <= numYears; year++) {
21         interest = (initialDeposit * annualInterestRate / 100);
22         initialDeposit += interest;
23         printf("%d년: 예금액= %.2lf, 이자= %.2lf\n", year, initialDeposit, interest);
24     }
25
26     return 0;
27 }
28 }
```

변수 year를 1부터 numYears까지
증가시키며 반복

현재 예금에 대한 이자를
계산하여 변수 interest에
저장

현재 예금에 이자를 더하여
새로운 예금액을 계산

4. for 문

LAB 암스트롱 수 구하기

■ 실행 결과

```
최대 범위를 입력하세요. : 900
최대 범위 내의 암스트롱 수를 찾습니다.
1 2 3 4 5 6 7 8 9 153 370 371 407
```

$$n = a^3 + b^3 + c^3$$

■ 조건

- 최대 범위를 입력받는다.
- 반복문을 사용하여 1부터 최대 범위까지의 숫자를 검사한다.
- 각 자리의 수를 분리하여 세제곱하고 그 값을 더한다.
- 더한 값이 원래의 수와 같으면 암스트롱 수로 판단한다.
- 암스트롱 수를 공백으로 구분하여 출력한다.

4. for 문

LAB 암스트롱 수 구하기

■ 힌트

- 주어진 범위 내에서 암스트롱 수를 찾기 위해 반복문과 조건문을 이용한다.

```
for (숫자 = 1; 숫자 <= 최대 범위; 숫자++)  
    // 1부터 사용자가 입력한 최대 범위(limit)까지 모든 숫자를 반복 확인한다.  
    // 각 자리의 수를 계산할 때 log10 함수를 사용한다.  
    while (숫자 > 0)  
        // 각 자리의 수를 분리한 후 n 제곱값을 계산할 때 pow() 함수를 사용한다.  
        if (합 == 숫자)  
            // 각 자리의 수를 n 제곱하여 합한 값이 원래 수와 같은지 확인한다.  
            // 계산된 합이 원래 수와 같다면 암스트롱 수로 판별하여 출력한다.
```

4. for 문

LAB 암스트롱 수 구하기

문제 해결

LAB_6-3.c

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03 #include <math.h>
04
05 int main()
06 {
07     int limit, num, sum, temp, n, digit; // 변수 선언
08
09     printf("최대 범위를 입력하세요. : ");
10     scanf("%d", &limit);
11
12     printf("최대 범위 내의 암스트롱 수를 찾습니다.\n");
13     for (num = 1; num <= limit; num++) {
14         sum = 0;
15         temp = num;
16         n = log10(num) + 1; // 숫자의 자릿수를 계산하여 변수 n에 저장
17
18         while (temp > 0) {
19             digit = temp % 10;
20             sum += pow(digit, n); // 현재 자릿수를 n 제곱한 값을 sum에 누적
21             temp /= 10;
22         }
23
24         if (sum == num) { // 현재 숫자가 암스트롱 수인지 확인
25             printf("%d ", num);
26         }
27     }
28     return 0;
29 }
```

Section 6.5

분기문

5. 분기문

■ break 문

- 제어 흐름 문장 중 하나로, 반복문 또는 switch 문에서 현재 실행 중인 코드를 즉시 종료하고 해당 블록 바로 다음의 코드로 제어 전달
 - 반복문에서의 break 문
 - 해당 반복문의 실행을 즉시 중단하고 반복문 바로 다음의 코드로 제어 이동
 - switch 문에서의 break 문
 - case 문이 실행될 때 break 문이 등장하면 해당 case 문을 즉시 중단

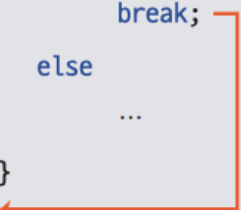
for 문

```
for (;;) {  
    ...  
    if (condition)  
        break;  
}
```



while 문

```
while () {  
    ...  
    if (condition)  
        break;  
    else  
        ...  
}
```



switch 문

```
switch () {  
    case value1:  
        ...  
        break;  
    case value2:  
        ...  
        break;  
}
```



5. 분기문

예제 6-12 for 문에서 break 문 사용하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int sum = 0;
07     for (int i = 1; i <= 100; i++){
08         sum = sum + i;
09         if (sum > 50){
10             printf("1부터 %d까지의 합은 %d입니다.\n", i, sum);
11             break;
12         }
13     }
14     return 0;
15 }
```

i의 값을 1부터 100까지 1씩 증가시키며 반복

sum 값이 50을 초과하면 아래 코드 실행

for 문을 즉시 종료

1부터 10까지의 합은 55입니다.

5. 분기문

예제 6-13 while 문에서 break 문 사용하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int sum = 0;
07     int input;
08     while (1) {
09         printf("숫자를 입력하세요 (-1을 입력하면 종료): ");
10         scanf("%d", &input);
11
12         if (input == -1) {
13             break;
14         }
15         sum += input;
16     }
17     printf("입력된 숫자의 합계는 %d입니다.\n", sum);
18
19     return 0;
20 }
21 }
```

조건이 항상 참이므로 무한 반복

사용자가 입력한 값이 -1인지 확인

사용자가 -1을 입력하면 즉시 종료

사용자가 입력한 숫자를 sum에 더하여 합계를 갱신

```
숫자를 입력하세요 (-1을 입력하면 종료): 75
숫자를 입력하세요 (-1을 입력하면 종료): 11
숫자를 입력하세요 (-1을 입력하면 종료): -1
입력된 숫자의 합계는 86입니다.
```

5. 분기문

■ continue 문

- 반복문 내에서 사용되며, 현재의 반복 주기를 즉시 종료하고 다음 반복 주기로 이동
- 해당 반복주기에서 continue 문 이후의 코드 실행 없음

▪ continue 문의 동작 원리

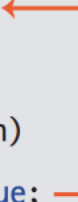
for 문

```
for (초기화; 조건식; 증감식) {  
    ...  
    if (condition)  
        continue;  
    printf("%d\n", i);  
}
```



while 문

```
while (조건식) {  
    ...  
    if (condition)  
        continue;  
    else  
        ...  
}
```



5. 분기문

예제 6-14 for 문에서 continue 문 사용하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     for (int i = 1; i <= 10; i++) {
07         if (i % 2 != 0) {
08             continue;
09         }
10         printf("%d\n", i);
11     }
12     return 0;
13 }
```

i의 값을 1부터 10까지 1씩 증가시키며 반복

i의 값을 2로 나눈 나머지가 0이 아닌지 확인

i의 값이 홀수이면 다음 반복 주기를 실행

i의 값이 홀수일 때 실행되지 않음



2
4
6
8
10

5. 분기문

예제 6-15 while 문에서 continue 문 사용하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int sum = 0;
07     int input;
08
09     while (1) {           // 조건이 항상 참이므로 무한 반복
10         printf("양수를 입력하세요 (-1을 입력하면 종료): ");
11         scanf("%d", &input);
12
13         if (input == -1) { // 사용자가 입력한 값이 -1인지 확인
14             break;        // 사용자가 -1을 입력하면 즉시 종료
15         }                // 사용자가 입력한 숫자가 음수인지 확인
16         if (input < 0) {
17             printf("음수는 무시됩니다.\n");
18             continue;     // 음수가 입력되면 sum += input; 구문을 건너뛰고 다음 반복을 이어감
19         }
20
21         sum += input;
22     }
23     printf("입력된 양수의 합계는 %d입니다.\n", sum);
24
25     return 0;
26 }
```

```
양수를 입력하세요 (-1을 입력하면 종료): 45
양수를 입력하세요 (-1을 입력하면 종료): -17
음수는 무시됩니다.
양수를 입력하세요 (-1을 입력하면 종료): 15
양수를 입력하세요 (-1을 입력하면 종료): -1
입력된 양수의 합계는 60입니다.
```

5. 분기문

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

() 문은 특정 조건에서 반복을 완전히 중단할 때 사용하고, () 문은 특정 조건에서 해당 반복 주기를 건너뛰는 때 사용한다.

goto 문

한 번 더 체크

goto 문

프로그램의 실행 흐름을 지정된 레이블로 강제로 전송하는 구문

goto 문 사용하기

ex_goto.c

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int i;
07
08     for (i = 0; i <= 10; i++) {
09         if (i == 6) {
10             goto end_loop;
11         }
12         printf("%d ", i);
13         printf("\n");
14     }
15
16     end_loop:
17     printf("반복이 끝난 후 i 값은 %d\n", i);
18
19     return 0;
20 }
```

goto 키워드 + 이동할 레이블 지정

해당 레이블로 이동하여 프로그램 종료

```
0
1
2
3
4
5
반복이 끝난 후 i 값은 6
```

목차

7.1 함수의 개요

7.2 함수 선언과 정의

7.3 표준 라이브러리 함수

학습목표

- C 언어에서 쓰이는 함수의 개념을 이해한다.
- 사용자 정의 함수와 매개변수, 반환값에 대해 알아본다.
- 표준 라이브러리 함수와 사용자 정의 함수에 대해 알아본다.

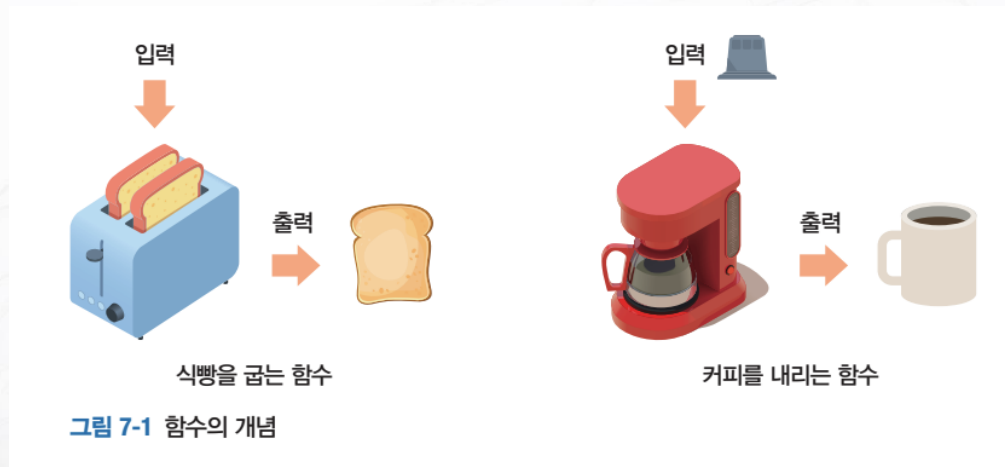
Section 7.1

함수의 개요

1. 함수의 개요

■ 함수의 개념

- 프로그램에서 특정 작업을 수행하도록 정의된 독립적인 단위
- 일련의 명령어와 연산으로 이루어진 독립적인 코드 블록
- 주어진 매개변수 입력을 바탕으로 특정 작업을 수행하고 그 결과를 반환하는 일련의 절차나 규칙
- 동일한 입력에 대해 같은 작업을 여러 번 수행할 수 있다는 것이 장점



1. 함수의 개요

■ 함수의 개념

- 정수형 변수 x의 값 25를 제공하여 변수 square에 저장하는 코드

```
int x = 25;
int square = 0;
square = x * x;
printf("square = %d ", x);
```

```
int x = 25;
int square = 0;
square = x * x;
printf("square = %d ", x);
```

... 다른 코드가 있음 ...

```
int x = 25;
int square = 0;
square = x * x;
printf("square = %d ", x);
```

반복 사용

... 다른 코드가 있음 ...

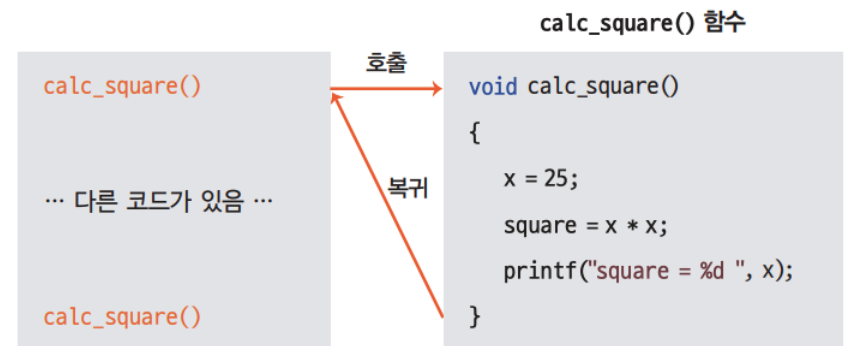


그림 7-2 함수의 활용 예

1. 함수의 개요

■ 함수의 장점

■ 코드의 재사용 용이

- 특정 작업을 수행하는 코드를 함수로 만들면 필요할 때마다 여러 번 호출 가능
- 동일한 코드를 여러 번 작성하지 않고도 쉽게 재사용 가능

■ 모듈화

- 함수 사용 시 코드를 논리적인 단위의 모듈로 분리 가능
- 각 부분의 기능과 책임이 명확해져 해당 기능이 필요할 때마다 재사용 가능

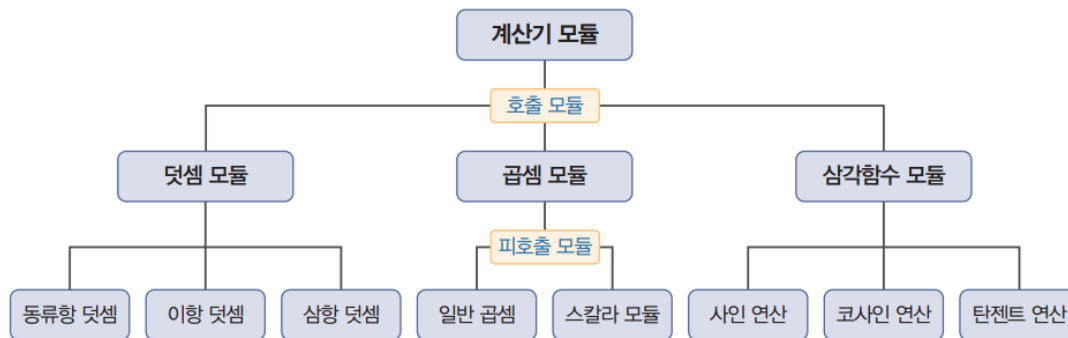


그림 7-3 함수 모듈화의 구성

1. 함수의 개요

■ 함수의 장점

■ 코드의 가독성 향상

- 하나의 특정 작업을 수행하므로 코드의 명확성과 구조가 개선

■ 유지·보수의 용이

- 특정 기능에 문제 발생 시 쉽게 발견 가능하고, 해당 함수만 수정 가능

■ 추상화

■ 코드의 일관성 유지

■ 매개변수와 반환값을 통한 데이터 반환

- 매개변수를 통해 데이터를 전달받고, 처리된 결과값을 반환하여 모듈 간 데이터의 흐름을 명확하게 관리하고 조작 가능

1. 함수의 개요

■ 함수의 종류

- C 언어에서 제공하는 함수와 사용자가 직접 만드는 함수로 구분
 - 표준 라이브러리 함수
 - » printf(), scanf() 등
 - » 사용자가 필요로 하는 다양한 기능을 지원
 - 사용자 함수
 - » 표준 라이브러리 함수에서 지원하지 않는 기능을 사용자가 각 프로그램에 맞게 직접 함수 생성



1. 함수의 개요

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

함수는 크게 C 언어에서 제공하는 () 함수와 사용자가 직접 만든 () 함수로 구분된다.

2 하나의 큰 프로그램을 독립적인 단위나 구성 요소로 분리하는 과정을 무엇이라고 하는가?

- ① 모듈화 ② 디버깅 ③ 추상화 ④ 인자

Section 7.2

함수 선언과 정의

2. 함수 선언과 정의

■ 사용자 정의 함수

- 프로그래머가 특정 작업을 수행하기 위해 직접 만드는 함수

■ 함수 정의

- 사용자 정의 함수를 구성하는 기본 요소
 - 함수명, 매개변수, 반환형, 함수 몸체

| Syntax | 함수 정의

```
반환형 함수명 (매개변수 1, 매개변수 2)
{
    // 함수 몸체
    // 실행될 문장
}
```

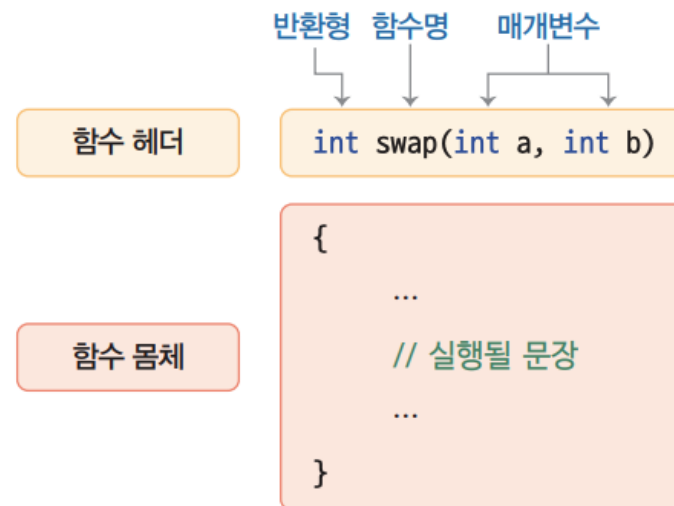


그림 7-5 사용자 정의 함수의 구성

2. 함수 선언과 정의

■ 사용자 정의 함수

■ 함수 정의

- 함수명

- 함수를 호출할 때 사용하는 이름인 함수명은 식별자 규칙에 따라 작성

- 매개변수

- 함수가 호출될 때 받을 입력값의 형태와 개수를 지정하는 '형식적인' 변수

```
int example (int x, char c)      // 2개 이상의 매개변수는 쉼표로 구분
int sample(void) or int sample() // 매개변수가 없을 때는 void를 사용
```

- 반환형

- 함수 수행이 종료된 후 함수를 호출한 곳으로 되돌아갈 때 반환되는 자료형

- 함수 몸체

- 함수가 수행하는 작업에 필요한 문장이 포함되며, 중괄호 안에 작성

2. 함수 선언과 정의

■ 사용자 정의 함수

■ 함수 선언

- 함수명, 반환형, 매개변수의 자료형을 컴파일러에 알려주는 역할
- 프로그램 내 함수의 실제 본체와는 별개로 해당 함수의 기본 정보를 컴파일러에 알리기 위한 것

```
#include <stdio.h>

int example (int x, char c);    // 함수 선언

int example (int x, char c) {    // 함수 정의
    // 실행될 코드
}
```

2. 함수 선언과 정의

■ 사용자 정의 함수

■ 함수 호출

- 괄호 안에 매개변수의 자료형과 개수, 반환 받을 변수의 자료형을 정확히 작성
- 함수가 호출되기 전까지는 함수 안의 코드가 실행되지 않으며, 함수를 호출하면 현재 실행 중인 코드가 잠시 중단되고, 호출된 함수로 이동하여 함수의 실행 코드를 순차적으로 실행

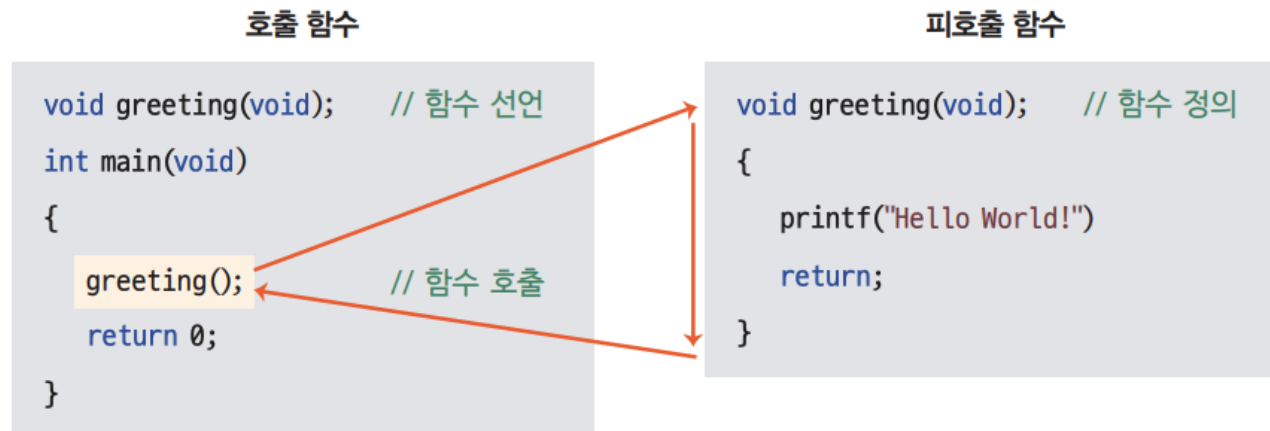


그림 7-6 호출 함수와 피호출 함수의 관계

2. 함수 선언과 정의

예제 7-1 사용자 정의 함수를 사용하여 삼각형의 넓이 계산하기

```
01 #include <stdio.h>
02 double triangleArea(double base, double height);
03
04 double triangleArea(double base, double height)
05 {
06     return (base * height) / 2.0;
07 }
08
09 int main()
10 {
11     double base = 10.0, height = 5.0;
12     double area = 0.0;
13
14     area = triangleArea(base, height);
15
16     printf("삼각형의 밑변이 %.2lf, 높이가 %.2lf일 때\n", base, height);
17     printf("삼각형의 넓이는 %.2lf입니다.\n", area);
18
19     return 0;
20 }
```

사용자 정의 함수 선언

함수의 반환값

triangleArea() 함수를 호출하여 base와 height의 값을 전달하고, 반환되는 삼각형의 넓이를 변수 area에 저장

삼각형의 밑변이 10.00, 높이가 5.00일 때
삼각형의 넓이는 25.00입니다.

2. 함수 선언과 정의

예제 7-2 사용자 정의 함수를 사용하여 팩토리얼 계산하기

```
01 #include <stdio.h>
02 long factorial(int n);
03
04 long factorial(int n)
05 {
06     long fac = 1;
07     for (int i = n; i >= 1; i--)
08         fac = fac * i;
09     return fac;
10 }
11
12 int main()
13 {
14     int number = 5;
15     long result = factorial(number);
16
17     printf("%d! = %ld\n", number, result);
18
19     return 0;
20 }
```

사용자 정의 함수 선언

n부터 1까지를 역순으로 반복하며
변수 fac에 차례대로 곱하는 for 문

factorial() 함수를 호출하여 number의 팩토리얼
값을 계산하고, 그 결과를 변수 result에 저장

5! = 120

2. 함수 선언과 정의

■ 함수의 형태

■ 매개변수와 반환값이 없는 함수

- 호출 시 전달하는 인자가 없고, 호출된 함수의 실행이 완료된 후 반환값을 받지 않는 구조

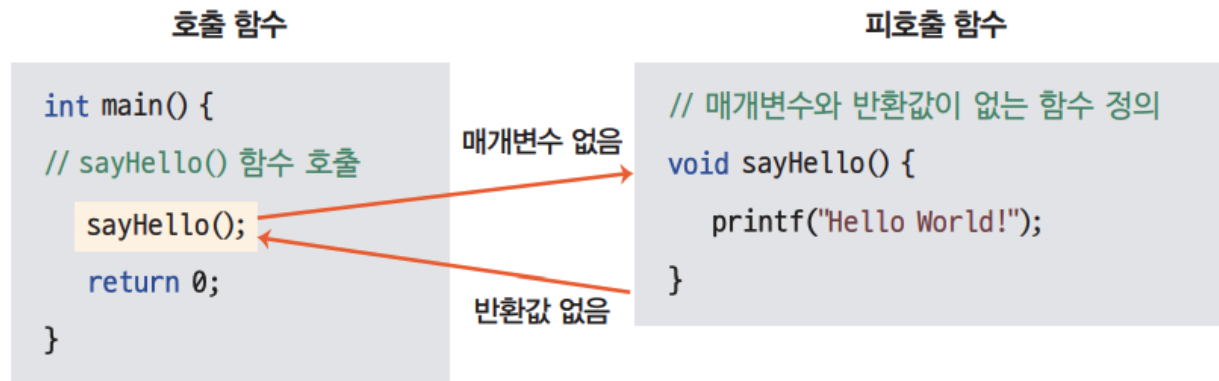


그림 7-7 매개변수와 반환값이 없는 함수의 구조

2. 함수 선언과 정의

예제 7-3 매개변수와 반환값이 없는 함수 사용하기

```
01 #include <stdio.h>
02
03 void function1()
04 {
05     printf("매개변수도 없고 반환값도 없는 함수\n");
06     printf("매개변수: 없음\n");
07     printf("반환값 없음: void\n");
08 }
09
10 int main()
11 {
12     function1();    // 매개변수와 반환값이 없는 함수 호출
13 }
```

매개변수와 반환값이 없음

```
매개변수도 없고 반환값도 없는 함수
매개변수: 없음
반환값 없음: void
```

2. 함수 선언과 정의

■ 함수의 형태

■ 매개변수가 있고 반환값이 없는 함수

- 함수 호출 시 인자로 전달받은 값을 사용하여 작업을 수행하지만, 함수가 실행된 후 결과를 반환하지 않는 구조

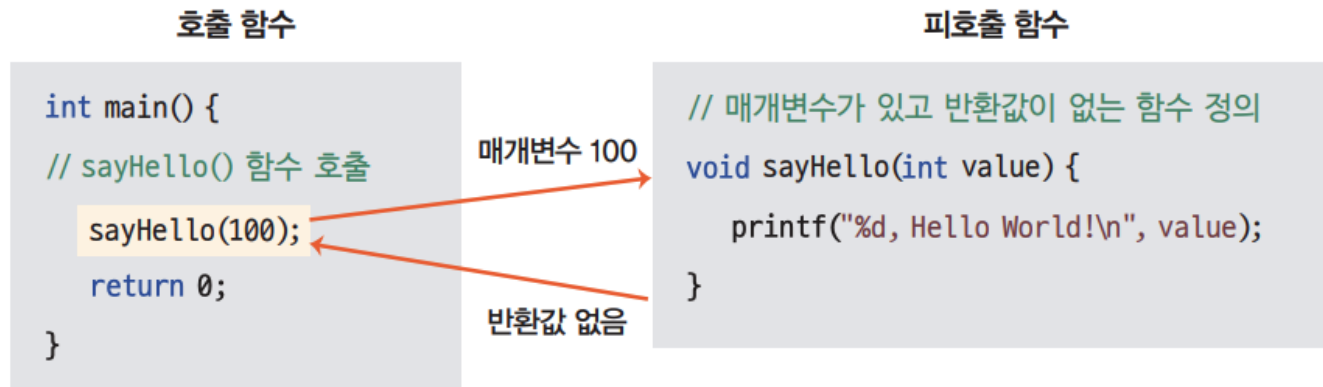


그림 7-8 매개변수가 있고 반환값이 없는 함수의 구조

2. 함수 선언과 정의

예제 7-4 매개변수가 있고 반환값이 없는 함수 사용하기

```
01 #include <stdio.h>
02
03 void function2(int x, int y)
04 {
05     printf("매개변수가 있고 반환값이 없는 함수\n");
06     printf("두 개의 매개변수: %d, %d\n", x, y);
07     printf("반환값 없음: void\n");
08 }
09
10 int main()
11 {
12     int a = 5, b = 7;
13     function2(a, b);
14
15     return 0;
16 }
```

매개변수 x, y가 있고 반환값이 없음

function2() 함수 호출 시 매개변수인 a, b의 값이 function2()의 x, y로 전달됨

매개변수가 있고 반환값이 없는 함수
두 개의 매개변수: 5, 7
반환값 없음: void

2. 함수 선언과 정의

■ 함수의 형태

■ 매개변수가 없고 반환값이 있는 함수

- 함수 호출 시 인자로 값을 전달받지 않으며, 함수가 실행된 후 결과를 반환하는 구조

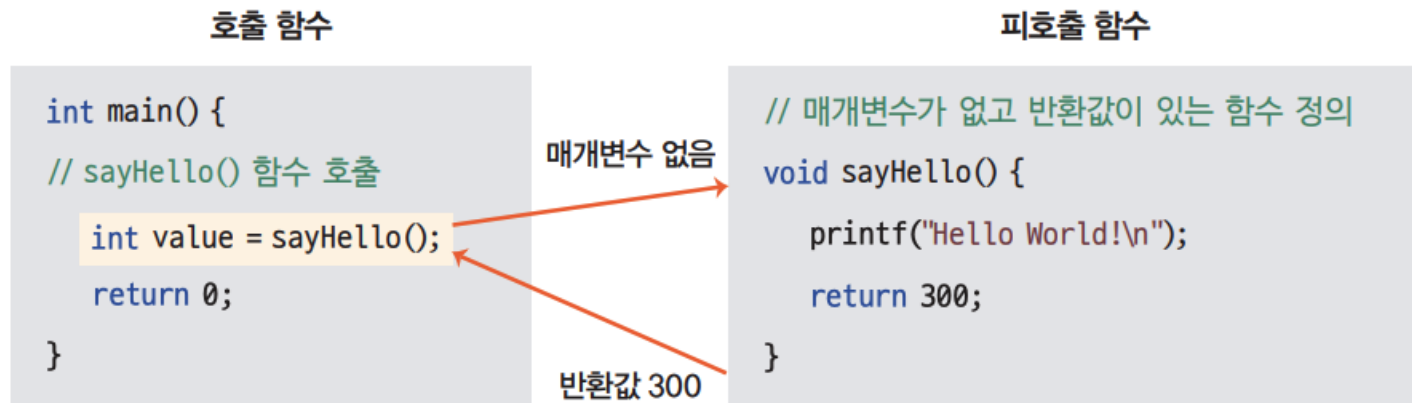


그림 7-9 매개변수가 없고 반환값이 있는 함수의 구조

2. 함수 선언과 정의

예제 7-5 매개변수가 없고 반환값이 있는 함수 사용하기

```
01 #include <stdio.h>
02
03 int function3()  함수 호출 시 매개변수를 전달받지 않음
04 {
05     printf("매개변수가 없고 반환값이 있는 함수\n");
06     printf("매개변수: 없음\n");
07     printf("반환값 있음: 42\n");
08
09     return 42;  반환값
10 }
11
12 int main()
13 {
14     int result3 = function3();  function3() 함수를 호출하고 반환값을 정수형 변수 result3에 저장
15     printf("function3의 반환값: %d\n", result3);
16
17     return 0;
18 }
```

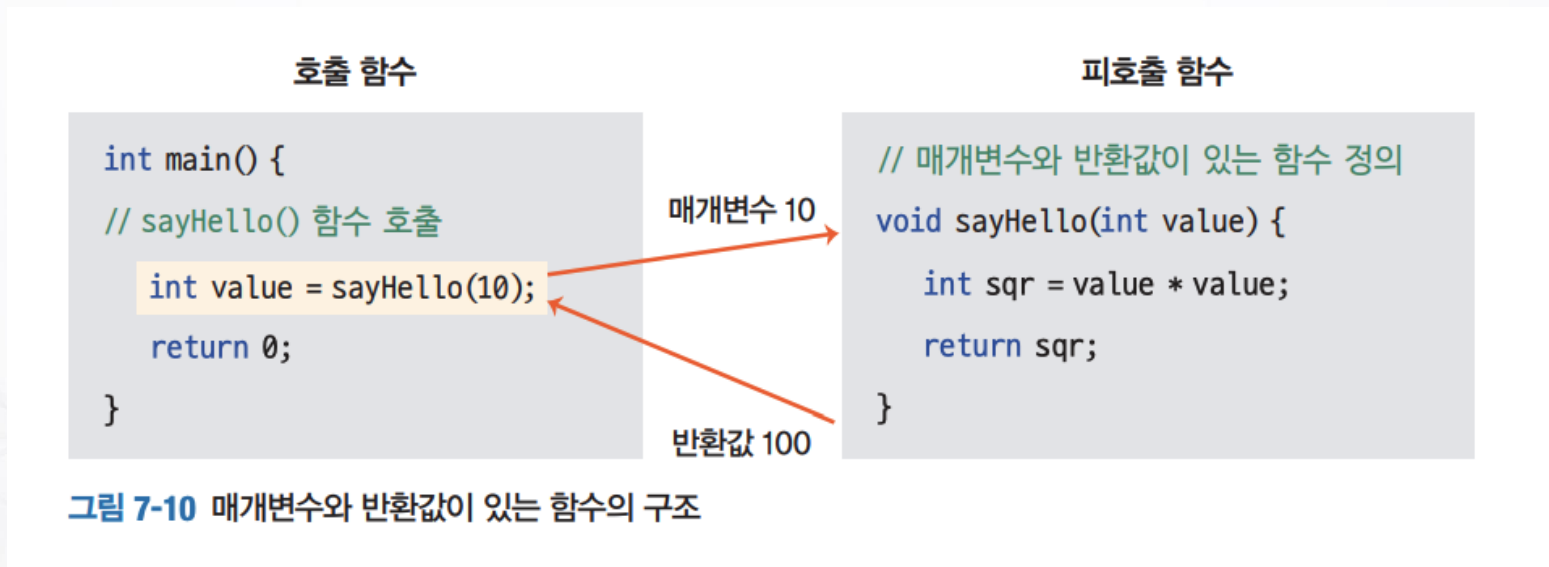
```
매개변수가 없고 반환값이 있는 함수
매개변수: 없음
반환값 있음: 42
function3의 반환값: 42
```

2. 함수 선언과 정의

■ 함수의 형태

■ 매개변수와 반환값이 없는 함수

- 인자로 전달받은 값을 활용하여 계산 또는 다른 연산을 수행한 후 결과를 반환하는 구조



2. 함수 선언과 정의

예제 7-6 매개변수와 반환값이 있는 함수 사용하기

```
01 #include <stdio.h>
02
03 int function4(int x, int y)
04 {
05     printf("두 개의 매개변수: %d, %d\n", x, y);
06     printf("반환값 있음: %d\n", x+y);
07
08     return x + y;
09 }
10
11 int main()
12 {
13     int a = 10, b = 6;
14     int result4 = function4(a, b);
15     printf("function4의 반환값: %d\n", result4);
16
17     return 0;
18 }
```

매개변수 x, y가 있음

반환값

함수 호출 시 매개변수인 a, b의 값이
function4()의 x, y에 전달되고 반환
값을 정수형 변수 result4에 저장

두 개의 매개변수: 10, 6
반환값 있음: 16
function4의 반환값: 16

2. 함수 선언과 정의

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① 함수를 ()할 때 사용하는 이름인 ()은 식별자 규칙에 따라 작성해야 하며, 코드
의 가독성을 고려하여 의미 있는 이름을 부여하는 것이 좋다.
- ② ()는 함수가 호출될 때 받을 입력값을 지정하기 위한 변수이다.

Section 7.3

표준 라이브러리 함수

3. 표준 라이브러리 함수

■ 라이브러리 함수의 사용법

- 함수 원형 선언
- 일반적으로 라이브러리 함수의 원형은 헤더 파일에 포함

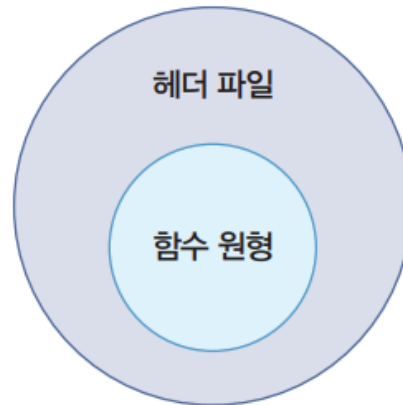


그림 7-11 헤더 파일과 함수 원형의 관계

3. 표준 라이브러리 함수

■ 난수 함수

▪ rand() 함수

- 헤더 파일에 정의되어 있는 rand() 함수는 호출할 때마다 0부터 RAND_MAX 사이의 무작위 정수를 반환

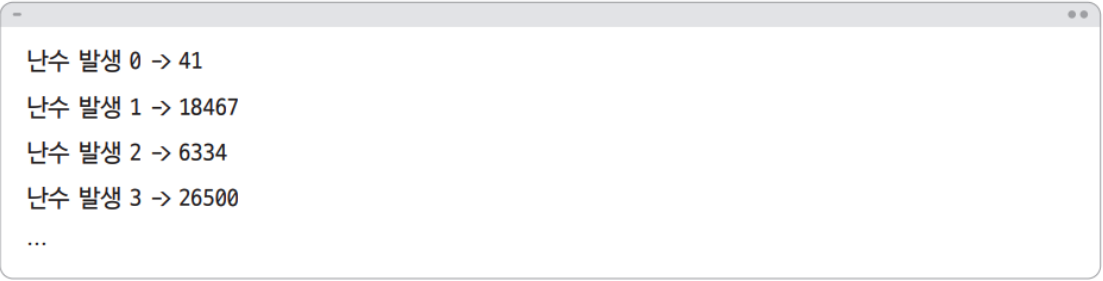
```
rand();
```

- 운영체제에 따라 다를 수 있지만 RAND_MAX는 일반적으로 정수 32767을 의미

3. 표준 라이브러리 함수

예제 7-7 rand() 함수를 사용하여 난수 생성하기 1

```
01 #include <stdio.h>
02 #include <stdlib.h>
03
04 int main()
05 {
06     int i;
07     for (i = 0; i < 10; i++) {    // 0~9까지 반복
08         printf("난수 발생 %d -> %d\n", i, rand());
09     }
10
11     return 0;
12 }
```



```
난수 발생 0 -> 41
난수 발생 1 -> 18467
난수 발생 2 -> 6334
난수 발생 3 -> 26500
...
```

3. 표준 라이브러리 함수

예제 7-8 rand() 함수를 사용하여 난수 생성하기 2

```
01 #include <stdio.h>
02 #include <stdlib.h>
03
04 int main()
05 {
06     int i;
07     for (i = 0; i < 10; i++) {
08         printf("%d ", rand() % 100);
09     }
10
11     printf("\n");
12
13     for (i = 0; i < 10; i++) {
14         printf("%d ", 1 + rand() % 100);
15     }
16
17     return 0;
18 }
```

rand() % 100은 0부터 99 사이의 값을 반환

1 + rand() % 100은 1부터 100 사이의 값을 반환

```
41 67 34 0 69 24 78 58 62 64
6 46 82 28 62 92 96 43 28 37
```

3. 표준 라이브러리 함수

■ 난수 함수

▪ srand() 함수

- rand() 함수에 의해 생성되는 난수 시퀀스의 시드를 설정하는 데 사용
- 시드를 설정하면 rand() 함수가 의사 난수를 생성할 때 시드 값을 기준으로 균등하게 분포된 난수인 난수 시퀀스를 시작

```
srand();
```

3. 표준 라이브러리 함수

예제 7-9 srand() 함수를 사용하여 난수 생성하기 1

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <time.h>           // 날짜와 시간 관련 기능을 제공하는 헤더 파일
04
05 int main()
06 {
07     srand(time(NULL));       // 현재 시간을 시드 값으로 설정
08     printf("%d\n", rand());  // 난수 출력
09
10     return 0;
11 }
```

12271

실행 시 값이 달라짐

3. 표준 라이브러리 함수

예제 7-10 srand() 함수를 사용하여 난수 생성하기 2

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <time.h>
04
05 int main()
06 {
07     int i;
08
09     srand(time(NULL));
10
11     for (i = 0; i < 9; i++) {
12         printf("시드 값 적용 난수 발생 %d - %d\n", i, rand());
13     }
14
15     return 0;
16 }
```

현재 시간을 시드 값으로 설정하여 난수 발생기를 초기화

0부터 RAND_MAX 사이의
난수를 생성 및 반환

```
시드 값 적용 난수 발생 0 - 13182
시드 값 적용 난수 발생 1 - 23258
시드 값 적용 난수 발생 2 - 7562
시드 값 적용 난수 발생 3 - 6433
...
```

실행 시 값이 달라짐

3. 표준 라이브러리 함수

예제 7-11 srand() 함수를 사용하여 난수 생성하기 3

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <time.h>
04
05 int main()
06 {
07     int i;
08     srand(time(NULL));
09
10     for (i = 0; i < 9; i++) {
11         printf("시드 값 적용 난수 발생 %d - %d\n", i, 1 + rand() % 100);
12     }
13
14     return 0;
15 }
```

현재 시간을 시드 값으로 설정하여 난수 발생기를 초기화

1~100까지의 난수 생성

```
시드 값 적용 난수 발생 0 - 33
시드 값 적용 난수 발생 1 - 58
시드 값 적용 난수 발생 2 - 86
시드 값 적용 난수 발생 3 - 5
...
```

실행 시 값이 달라짐

3. 표준 라이브러리 함수

■ 수학 함수

- 특정 수학적 연산이나 계산으로 수행하는 함수
- C 언어의 표준 라이브러리에 포함된 수학 함수는 대부분 실수 연산에 초점

표 7-1 수학 함수

구분	함수	설명	구분	함수	설명
삼각 함수	$\sin(x)$	x의 사인 값을 반환한다.	지수 함수와 로그 함수	$\exp(x)$	e^x 값을 반환한다.
	$\cos(x)$	x의 코사인 값을 반환한다.		$\log(x)$	x의 자연로그 값을 반환한다.
	$\tan(x)$	x의 탄젠트 값을 반환한다.		$\log_{10}(x)$	x의 상용로그 값을 반환한다.
역삼각 함수	$\asin(x)$	x의 역사인 값을 반환한다.		$\text{pow}(x, y)$	x^y 값을 반환한다.
	$\acos(x)$	x의 역코사인 값을 반환한다.		$\text{sqrt}(x)$	x의 제곱근 값을 반환한다.
	$\text{atan}(x)$	x의 역탄젠트 값을 반환한다.	반올림 함수와 나머지 함수	$\text{ceil}(x)$	x 이상의 최소 정수를 반환한다.
	$\text{atan2}(y, x)$	(y, x)에 대한 역탄젠트 값을 반환한다.		$\text{floor}(x)$	x 이하의 최대 정수를 반환한다.
쌍곡선 함수	$\sinh(x)$	x의 하이퍼볼릭 사인 값을 반환한다.	기타 함수	$\text{fmod}(x, y)$	x를 y로 나눈 나머지를 반환한다.
	$\cosh(x)$	x의 하이퍼볼릭 코사인 값을 반환한다.		$\text{fabs}(x)$	x의 절댓값을 반환한다.
	$\tanh(x)$	x의 하이퍼볼릭 탄젠트 값을 반환한다.		$\text{hypot}(x, y)$	직각 삼각형의 빗변 길이를 반환한다.
				$\text{ldexp}(x, \text{exp})$	x에 2의 exp 제곱을 곱한 결과를 반환한다.

3. 표준 라이브러리 함수

예제 7-12 `sin()`, `cos()`, `tan()` 함수 사용하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03 #include <math.h>
04
05 int main()
06 {
07     double angle, sin_val, cos_val, tan_val;
08
09     printf("각도(라디안) 입력: ");
10     scanf("%lf", &angle);    // 각도(라디안) 입력 요청
11
12     // sin, cos, tan 값 계산
13     sin_val = sin(angle);
14     cos_val = cos(angle);
15     tan_val = tan(angle);
16
17     printf("sin(%.2lf) = %lf\n", angle, sin_val);
18     printf("cos(%.2lf) = %lf\n", angle, cos_val);
19     printf("tan(%.2lf) = %lf\n", angle, tan_val);
20
21     return 0;
22 }
```

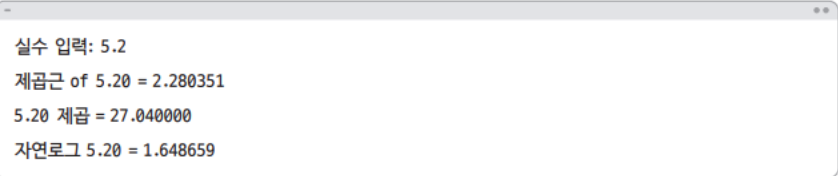
입력받은 각도의 사인, 코사인, 탄젠트 값을
각각 sin_val, cos_val, tan_val에 저장

```
각도(라디안) 입력: 5
sin(5.00) = -0.958924
cos(5.00) = 0.283662
tan(5.00) = -3.380515
```

3. 표준 라이브러리 함수

예제 7-13 sqrt(), pow() 함수 사용하기

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03 #include <math.h>
04
05 int main()
06 {
07     double num, base, sqrt_val, pow_val, log_val;
08     printf("실수 입력: ");
09     scanf("%lf", &num);
10
11     sqrt_val = sqrt(num);    // sqrt() 함수로 제곱근 계산
12     pow_val = pow(num, 2);   // pow() 함수로 제곱 계산
13
14     // log() 함수로 자연로그 계산(입력값이 양수여야 함)
15     if (num <= 0) {
16         printf("0 이하의 값에 대해 로그를 계산할 수 없습니다.\n");
17     }
18     else {
19         log_val = log(num);
20     }
21     printf("제곱근 of %.2lf = %lf\n", num, sqrt_val);
22     printf("%.2lf 제곱 = %lf\n", num, pow_val);
23
24     if (num > 0) {
25         printf("자연로그 %.2lf = %lf\n", num, log_val);
26     }
27
28     return 0;
29 }
```



실수 입력: 5.2
제곱근 of 5.20 = 2.280351
5.20 제곱 = 27.040000
자연로그 5.20 = 1.648659

3. 표준 라이브러리 함수

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① C 언어에서 자체적으로 제공하는 함수를 ()라고 한다. 일반적으로 라이브러리 함수의 원형은 ()에 포함되어 있다.
- ② 난수를 생성하는 대표적인 함수는 ()와 ()이다.
- ③ `<math.h>`에서 제공하는 삼각 함수는 (), (), ()이다.

3. 표준 라이브러리 함수

LAB 주사위 게임 만들기

■ 실행 결과

주사위 숫자 맞추기 게임을 시작합니다.
주사위를 던져 나올 숫자를 맞춰보세요 (1에서 6까지): 3
주사위 결과: 3
축하합니다! 숫자를 맞혔습니다.

주사위를 던져 나올 숫자를 맞춰보세요 (1에서 6까지): 2
주사위 결과: 4
아쉽습니다. 다시 시도하세요.



3. 표준 라이브러리 함수

LAB 주사위 게임 만들기

■ 조건

- 다음과 같은 함수를 작성한다.
 - 1부터 6까지의 무작위 숫자를 생성하는 함수
 - 사용자로부터 숫자를 입력받는 함수
 - 사용자가 예상한 숫자와 주사위를 던져서 나오는 숫자가 일치하는지를 조건문으로 판단하는 함수
- 사용자가 0을 입력할 때까지 게임이 계속 반복된다.

■ 힌트

- 주사위를 던져서 나오는 숫자는 rand() 함수로 구현할 수 있다.
- main() 함수에서 srand(time(NULL))로 난수 발생기를 초기화하면 매번 다른 결과를 얻을 수 있다. #include 와 #include 헤더 파일을 추가한다.

3. 표준 라이브러리 함수

LAB 주사위 게임 만들기

문제 해결

LAB_7-1.c

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>    // 표준 입출력 관련 기능을 제공하는 헤더 파일
03 #include <stdlib.h>    // 표준 라이브러리 함수
04 #include <time.h>      // 날짜와 시간 관련 기능을 제공하는 헤더 파일
05
06 int throwDice();
07 int getUserGuess();    // 사용자 정의 함수 선언
08 void playGuessingGame();
09
10 int throwDice() {
11     return rand() % 6 + 1;
12 }
13
14 int getUserGuess() {
15     int guess;
16     printf("주사위를 던져 나올 숫자를 맞춰보세요 (1에서 6까지): ");
17     scanf("%d", &guess);
18
19     return guess;
20 }
21
```

1부터 6까지의 무작위 숫자 반환

입력받은 숫자를 반환하는 함수

3. 표준 라이브러리 함수

LAB 주사위 게임 만들기

```
22 void playGuessingGame() {
23     int userGuess, diceResult;
24
25     do {
26         userGuess = getUserGuess();
27         diceResult = throwDice();
28         printf("주사위 결과: %d\n", diceResult);
29         if (userGuess == diceResult) {
30             printf("축하합니다! 숫자를 맞혔습니다.\n");
31         }
32         else {
33             printf("아쉽습니다. 다시 시도하세요.\n");
34         }
35     } while (userGuess != 0);
36 }
37
38 int main() {
39
40     srand(time(NULL));
41
42     printf("주사위 숫자 맞추기 게임을 시작합니다.\n");
43
44     playGuessingGame();
45
46     printf("게임 종료\n");
47
48     return 0;
49 }
```

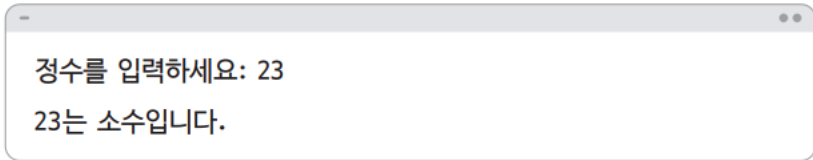
사용자가 0을 입력할 때까지 게임을 반복하기 위한 do~while 문

난수 발생 초기화

3. 표준 라이브러리 함수

LAB 소수 판별기 만들기

■ 실행 결과



```
정수를 입력하세요: 23
23는 소수입니다.
```

■ 조건

- 정수를 입력받는다.
- 입력받은 정수가 소수인지 판별하는 사용자 정의 함수를 작성하는데, 소수이면 1을 반환하고 아니면 0을 반환한다.
- 소수 여부에 따라 결과를 출력한다.

■ 힌트

- 소수를 판별할 때는 제곱근을 계산하는 `sqrt()` 함수를 사용한다. 어떤 수를 제곱한 값은 해당 수의 제곱근이며, 예를 들어 4의 제곱근은 2이다. `sqrt()` 함수는 헤더 파일에 있다.



3. 표준 라이브러리 함수

LAB 소수 판별기 만들기

문제 해결

LAB_7-2.c

```
01 #pragma warning(disable:4996)
02 #include <stdio.h>           // 표준 입출력 관련 기능을 제공하는 헤더 파일
03 #include <math.h>            // 수학 함수를 제공하는 헤더 파일
04
05 int isPrime(int num);        // 소수를 판별하는 사용자 정의 함수 선언
06
07 int isPrime(int num) {
08
09     if (num <= 1) return 0;   num이 1보다 작거나 같으면 0을 반환
10
11     for (int i = 2; i <= sqrt(num); ++i) {
12         if (num % i == 0) return 0; 2부터 입력된 정수의 제곱근까지 반복 수행
13     }
14
15     return 1;
16 }
```

3. 표준 라이브러리 함수

LAB 소수 판별기 만들기

```
17
18 int main()
19 {
20     int num;
21
22     printf("정수를 입력하세요: ");
23     scanf("%d", &num);
24
25     if (isPrime(num)) {
26         printf("%d는 소수입니다.\n", num);
27     }
28     else {
29         printf("%d는 소수가 아닙니다.\n", num);
30     }
31
32     return 0;
33 }
```

isPrime() 함수의 매개변수로, 함수 내부에서 사용되는 지역 변수

반환값에 따라 소수 여부 메시지 출력

**3주차 강의가 끝났습니다,
모두 고생하셨습니다.**

